AI Image Generator

Technical Documentation, Workflows and Requirements

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

Contents

1. Project Overview 6

1.1 Purpose 6

1.2 Key capabilities 6

1.3 Target environment 7

1.4 Document map 7

2. Requirements 9

2.1 Functional requirements 9

2.2 Non-functional requirements 10

2.3 Hardware requirements 10

2.4 Software requirements 11

2.5 Model assets 11

2.6 Constraints and assumptions 12

3. System Architecture 13

3.1 Component overview 13

3.2 Generation pipeline 13

3.3 Data flow and storage 14

3.4 Design decisions 14

3.5 Concurrency model 15

4. Workflows 16

4.1 First-time setup 16

4.2 Daily use: generating an image 16

4.3 Choosing a model 17

4.4 Using LoRAs and your own checkpoints 17

4.5 Producing 4K and 8K output 17

4.6 Reviewing past generations 18

4.7 Adding a model or a LoRA 18

4.8 Managing models and LoRAs 18

4.9 Watching the machine: Overview, System and Logs 19

4.10 Batch generation from the command line 19

4.9 Upscaling an existing image 19

4.10 Enabling FLUX.1 19

4.11 Running in Docker 20

4.12 Stopping, restarting and backup 20

4.14 Generating a video clip 20

4.15 Generating a music track 20

4.13 Development workflow . . . 21
5. Module Reference . . . 22
5.1 app.py (about 640 lines) . . . 22
5.2 models.py (about 680 lines) . . . 22
5.3 registry.py (about 300 lines) . . . 23
5.4 downloads.py (about 480 lines) . . . 23
5.5 admin_ui.py (about 600 lines) . . . 24
5.6 ui_theme.py, ui_common.py, applog.py . . . 24
5.7 prompt_parser.py (315 lines) . . . 25
5.8 sysinfo.py (358 lines) . . . 25
5.9 upscaler.py (109 lines) . . . 25
5.10 gpu_info.py (97 lines) . . . 26
5.11 db.py (about 270 lines) . . . 26
5.12 generate.py (125 lines) . . . 26
5.14 media.py (about 330 lines) . . . 26
5.15 media_ui.py (about 420 lines) . . . 27
5.16 video_io.py (about 190 lines) . . . 27
5.13 Supporting files . . . 27
6. Models and Prompt Enhancement . . . 29
6.1 Built-in models . . . 29
6.2 Automatic selection . . . 30
6.3 Steps and guidance follow the model . . . 30
6.4 Prompt weighting and long prompts . . . 31
6.5 LoRAs . . . 31
6.6 Local checkpoints . . . 31
6.7 Z-Image Turbo, the default model . . . 31
6.8 Qwen-Image on 24 GB . . . 31
6.9 FLUX.1 schnell on 24 GB . . . 32
7. Prompt Language Reference . . . 33
7.1 Model and LoRA . . . 33
7.2 Size and shape . . . 33
7.3 Output resolution (upscaling) . . . 33
7.4 Quality and fidelity . . . 34
7.5 Style presets . . . 34
7.6 Batch and reproducibility . . . 34
7.7 Worked examples . . . 34

8. Database . . . 36
8.1 Overview . . . 36
8.2 Table: generations . . . 36
8.3 Example queries . . . 38
8.4 Access from Python . . . 38
9. Admin Panel . . . 39
9.1 Model editor and actions . . . 39
9.2 LoRA editor and actions . . . 39
9.3 Sources the Add tab understands . . . 40
9.4 System tiles . . . 40
9.5 System sections . . . 41
9.6 Reading the numbers . . . 41
9.7 Locking the panel . . . 41
10. Video and Music . . . 43
10.1 Overview . . . 43
10.2 The Video tab . . . 43
10.3 The Music tab . . . 44
10.4 Sharing one GPU . . . 44
10.5 How the models are loaded . . . 44
10.6 Files, thumbnails and the database . . . 44
10.7 Measured on the RTX 3090 . . . 44
10.8 Limits and tips . . . 45
11. Performance . . . 46
11.1 Generation by model (832x1216, prompt weighting on for SDXL) . . . 46
11.2 Generation (v1.0 measurements, 1024x1024 and 16:9, SDXL base) . . . 46
11.3 Upscaling . . . 46
11.4 Memory . . . 46
11.5 Startup and switching . . . 47
11.6 Tuning levers . . . 47
12. Deployment and Docker . . . 48
12.1 Image . . . 48
12.2 Compose service . . . 48
12.3 Host requirements . . . 48
12.4 Commands . . . 48
12.5 Differences inside the container . . . 48
13. Operations and Troubleshooting . . . 49

13.1 Starting and stopping . . . 49
13.2 Logs . . . 49
13.3 Known issues encountered during development and their fixes . . . 49
13.4 Troubleshooting guide . . . 50
13.5 Maintenance . . . 51
13.6 Security notes . . . 51
14. Change Log and Roadmap . . . 52
14.1 Change log . . . 52
14.2 Roadmap . . . 55
Appendix . . . 56
A. Project file tree . . . 56
B. requirements.txt . . . 56
C. Command cheat sheet . . . 57
D. Programmatic access to the running app . . . 57
E. Licences . . . 57

1. Project Overview

1.1 Purpose

AI Image Generator is a self-hosted application that turns a plain-English prompt into an image using Stable Diffusion XL, its best community fine-tunes, Z-Image Turbo, Qwen-Image or FLUX.1, into a short video clip using LTX-Video or Wan 2.1, or into a music track using MusicGen, running entirely on a local NVIDIA GPU. No data leaves the machine and there is no per-image cost.

The project was built on 21 and 22 October 2026 and grew through twelve increments:

1. A command-line script that runs Stable Diffusion XL on the GPU.
2. A local web interface so a prompt can be typed and the result viewed in a browser.
3. A prompt language that replaces sliders and dropdowns: size, ratio, quality, style, count and seed are all read from the prompt text.
4. Per-image telemetry (timings, VRAM, temperature, power, clocks) shown in the UI and stored in a SQLite database with a History tab.
5. A Real-ESRGAN upscaling stage so that "4k" or "8k" in the prompt produces a genuine 3840 or 7680 pixel image.
6. A model registry: six SDXL-family checkpoints, Z-Image Turbo and FLUX.1 schnell, automatic model choice by style, local checkpoints, LoRAs, prompt weighting and per-model samplers.
7. The Frazile AI studio interface: dark theme with pink-to-yellow accents, full-width layout, and an Upscale bar that appears after generation.
8. An Admin panel with live CPU, GPU, memory, storage, network and power telemetry, plus a Docker image for container deployment.
9. Z-Image Turbo as the default model, loaded shard by shard onto the GPU with a cached bf16 copy.
10. Starter LoRAs with a LoRAs box, and Qwen-Image (20B) in 4-bit with the Lightning 8-step LoRA.
11. Model and LoRA management in the Admin panel: a persistent registry, background downloads from Hugging Face, Civitai, URLs and local files, per-LoRA family and trigger words, settings, logs; plus quick-hint chips, live header, thumbnails and reuse actions in the Studio and History tabs.
12. Video and music: a Video tab (LTX-Video text- and image-to-video, Wan 2.1, frame-by-frame Real-ESRGAN upscaling to Full HD, 2K or 4K, soundtracks) and a Music tab (MusicGen), both recorded in History and managed from the same Admin panel.

1.2 Key capabilities

- Text-to-image generation with eight built-in models: Z-Image Turbo (the default), SDXL base 1.0, Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo, Playground v2.5, Animagine XL 3.1 and FLUX.1 schnell, all in 16-bit precision (FLUX in 4-bit on 24 GB cards).
- Models and LoRAs added from the Admin panel (Hugging Face repo or file, Civitai link, direct URL, local path or upload) are usable as soon as their background download finishes, without a restart; files dropped into models/checkpoints/ and models/loras/ by hand still work.
- Every model's defaults (steps, guidance, sampler, description, aliases, auto-pick styles, enabled, start-up) and every LoRA's family, trigger words and default weight are editable in the Admin panel and persisted in models/registry.json.
- Prompt-driven control of model, LoRAs, emphasis weighting, aspect ratio, exact resolution, orientation, quality, fidelity, art style, image count, seed, steps and guidance.
- Automatic model selection from style words (photorealistic goes to Juggernaut, anime to Animagine, painting to Playground, digital art to DreamShaper).

- Live interpretation line under the prompt box showing exactly what will be generated, with which model, before the button is pressed.
- Tiled Real-ESRGAN upscaling to 2K, 4K, 8K or any exact size, either from the prompt or afterwards from the Upscale bar, with automatic centre crop.
- Full telemetry per image: inference time, steps per second, peak VRAM, GPU temperature and power before and after, clocks, fan, performance state, driver and library versions.
- Persistent history in SQLite with search, aggregate statistics, and click-to-view of any past image with its full record, including later upscales.
- Admin panel with eight sub-tabs: Overview (system and activity tiles, per-model statistics), Models, LoRAs, Add, Downloads, System (nine live tiles and six detail sections), Settings (start-up model, tokens, housekeeping, admin lock) and Logs.
- Studio conveniences: quick-hint chips, a live header with loaded model, VRAM, temperature and running downloads, LoRA trigger-word reminders, Reuse seed and New variation buttons; History with model filter, thumbnail grid, reuse prompt (with or without seed) and delete.
- Command-line tools for batch generation with any model and LoRA, upscaling an existing image, querying the database and dumping system telemetry.
- A Dockerfile and docker-compose.yml for running the whole studio as a GPU container.
- Text-to-video and image-to-video with LTX-Video 2B (24 fps, 768x512 and five other sizes, 2 to 10 s) and text-to-video with Wan 2.1 1.3B (16 fps, 832x480), exported as H.264 MP4; every frame can be upscaled with Real-ESRGAN to Full HD, 2K or 4K and a generated music track can be laid under the clip.
- Text-to-music with MusicGen stereo medium and small (32 kHz stereo, 3 to 30 s, instrumental) and, for owners of a Hugging Face token, Stable Audio Open 1.0; tracks are saved as 16-bit WAV and shown with a waveform thumbnail in History.

1.3 Target environment

The application was developed and verified on the following machine. Other NVIDIA GPUs with at least 12 GB of VRAM will work for the SDXL-family models; see the Requirements section for the full matrix.

Item	Value
GPU	NVIDIA GeForce RTX 3090, 24 GB VRAM, compute capability 8.6, 82 SMs
CPU / RAM	AMD Ryzen 5 3500 (6 cores), 16 GB
Driver	610.88
OS	Windows 11 Pro 10.0.26300
Python	3.12.0
CUDA runtime (via PyTorch)	12.6
Project folder	C:\Users\frazi\Documents\apps\ai-image-gen
Repository	https://github.com/Fzhacking/ai-image-gen (private)

1.4 Document map

Section	Contents
2. Requirements	Functional and non-functional requirements, hardware and software matrix, model assets
3. Architecture	Components, pipeline diagram, data flow, design decisions, concurrency
4. Workflows	Setup, generation, choosing models, LoRAs, upscaling, history, adding models and LoRAs, Admin panel, Docker, maintenance
5. Module reference	Every source file, its public functions and their contracts

Section	Contents
6. Models	The model registry, automatic selection, samplers, prompt weighting, LoRAs, FLUX
7. Prompt language	Complete reference of words the parser understands
8. Database	Schema of generations.db, every column explained, example queries
9. Admin panel	Model and LoRA management, sources and downloads, settings and logs, every tile and section of the system panel
10. Video and music	The Video and Music tabs, their models, GPU sharing, files, database columns and measured figures
11. Performance	Measured timings and resource use on the RTX 3090 for every model
12. Deployment and Docker	Container image, compose file, volumes, GPU access, host requirements
13. Operations	Running, stopping, troubleshooting, known issues, backup, security
14. Change log and roadmap	What was built in which order, defects fixed, future work
Appendix	File tree, dependency list, command cheat sheet, API access, licences

2. Requirements

2.1 Functional requirements

ID	Requirement	Status
FR-01	Generate an image from a text prompt using a locally hosted diffusion model on the GPU.	Done
FR-02	Provide a browser interface where a prompt can be typed and the result is displayed.	Done
FR-03	Keep the model loaded between requests so repeat generations take seconds, not minutes.	Done
FR-04	Control aspect ratio, resolution, orientation, quality, fidelity, style, count and seed from words in the prompt, with no separate controls.	Done
FR-05	Show how the prompt was interpreted before generation starts.	Done
FR-06	Produce genuine 4K (3840 px) and 8K (7680 px) output when requested, and exact sizes such as 3840x2160.	Done
FR-07	Display, for every image, the GPU used, time taken, VRAM, temperature, power, clocks and software versions.	Done
FR-08	Persist every generation with its settings and telemetry in a database that can be browsed later.	Done
FR-09	Provide a History view with search, aggregate statistics and click-to-view of past images.	Done
FR-10	Save every generated image, native and upscaled, as a PNG file on disk.	Done
FR-11	Support a negative prompt describing what to avoid.	Done
FR-12	Provide command-line equivalents for generation, upscaling and database queries.	Done
FR-13	Offer several models with different strengths and let the prompt or a dropdown choose between them; switch models without restarting.	Done (v2.0)
FR-14	Choose a suitable model automatically from the style words in the prompt.	Done (v2.0)
FR-15	Accept user-supplied SDXL checkpoints and LoRAs from local folders.	Done (v2.0)
FR-16	Support emphasis weighting such as (word:1.3) and prompts longer than 77 tokens.	Done (v2.0)
FR-17	Upscale an already generated image from the UI without regenerating it, recording the result on the same history row.	Done (v2.0)
FR-18	Present the application in Frazile AI branding with a full-width layout.	Done (v2.0)
FR-19	Provide an Admin panel with live CPU, GPU, memory, storage, network, power and software information.	Done (v2.0)
FR-20	Run the whole application as a GPU-enabled Docker container.	Done, untested on the development machine (no Docker installed)
FR-21	Add models and LoRAs from the Admin panel: Hugging Face repos and files, Civitai links, direct URLs, local files and uploads, with background downloads and progress.	Done (v2.1)
FR-22	Edit any model's defaults and any LoRA's family, trigger words and weight; enable, disable, load, unload, delete and choose the start-up model from the Admin panel.	Done (v2.1)
FR-23	Record which family a LoRA was trained for, route Auto to a matching model and never apply a LoRA to the wrong family.	Done (v2.1)
FR-24	Show the application's own log and generation statistics per model in the Admin panel.	Done (v2.1)
FR-25	Protect destructive Admin actions with a confirmation and an optional password when the server is reachable from other machines.	Done (v2.1)
FR-26	Generate a short video clip from a text prompt, or from a text prompt plus a start image, with a choice of size, length, steps, guidance and seed, saved as an MP4 that browsers play.	Done (v2.2)

ID	Requirement	Status
FR-27	Upscale a generated clip frame by frame to Full HD, 2K or 4K with Real-ESRGAN, and lay a generated music track under it.	Done (v2.2)
FR-28	Generate a music track from a text prompt with a choice of length, guidance, temperature and seed, saved as WAV.	Done (v2.2)
FR-29	Manage video and music models like image models (registry, Admin editing, background download, load, unload, start-up) and record every clip and track in History with telemetry.	Done (v2.2)

2.2 Non-functional requirements

ID	Requirement	How it is met
NFR-01	Runs fully offline after the first model download.	Model weights cached in the Hugging Face cache; upscaler weights stored in models/.
NFR-02	A 1 megapixel image generates in under 15 seconds on an RTX 3090.	Measured 2.0 to 10.5 s depending on model, steps and size.
NFR-03	Peak VRAM stays under 16 GB so 16 GB cards can run every SDXL-family model.	Measured peak 13.2 GB at 832x1216 with prompt weighting; 10.5 GB at 1024x1024 without.
NFR-04	Upscaling must never exceed VRAM regardless of output size.	512 px tiles with 32 px overlap; memory is bounded per tile.
NFR-05	Switching models must free the previous model's VRAM.	ModelManager drops all references, runs gc and empties the CUDA cache; measured 6.58 GB to 0.01 GB resident.
NFR-06	The server binds to localhost only unless deliberately configured otherwise.	Gradio launched on 127.0.0.1:7860; the Docker image binds 0.0.0.0 inside the container only.
NFR-07	All persistent state is in plain files that can be backed up by copying.	PNG files in outputs/, SQLite file generations.db, weights in models/ and the HF cache.
NFR-08	The database schema can evolve without losing data.	db.connect() adds missing columns automatically.
NFR-09	The Admin panel must not slow the GPU work.	A refresh costs about 0.7 s after the first; folder sizes and thread counts are cached for 60 and 30 s.
NFR-10	Code is dependency-light and readable: no framework beyond Gradio and diffusers.	Fourteen Python modules, about 4,300 lines in total.
NFR-11	Downloads must not block generation or the interface.	Downloads run in a two-worker thread pool; the GPU work and the statistics refreshes use separate Gradio queues.
NFR-12	Admin edits must survive restarts and be inspectable.	models/registry.json, written atomically; the built-in list stays in code and only overrides are stored.
NFR-13	Video and music must share the single GPU with the image models without manual unloading.	media.prepare_for(): a video model and an image model never share the card; a music model stays next to an image model while VRAM allows; the image model reloads on the next image.
NFR-14	Video and audio export must not depend on a system ffmpeg install.	imageio-ffmpeg ships a static ffmpeg binary; video_io.py streams frames to it over a pipe so no clip is held in RAM.

2.3 Hardware requirements

Component	Minimum	Recommended	Verified on
GPU	NVIDIA, 12 GB VRAM, compute capability 7.0+ (SDXL family); 24 GB for Z-Image Turbo and FLUX	24 GB	RTX 3090, 24 GB
System RAM	16 GB	32 GB	16 GB
Disk	40 GB free (Z-Image Turbo 20 GB plus its 12 GB bf16 cache, 64 MB upscaler, Python environment, outputs)	120 GB: every SDXL model is 6.5 GB and FLUX is 33 GB	250 GB free, 71 GB used by models

Component	Minimum	Recommended	Verified on
CPU	Any x86-64	6 cores or more	Ryzen 5 3500

Cards below 12 GB can run the command-line script with CPU offload, but the web app assumes an SDXL pipeline fits in VRAM. FLUX on a card under 40 GB loads its transformer in 4-bit and its T5 encoder in 8-bit with per-model CPU offload.

2.4 Software requirements

Package	Version used	Purpose
Python	3.12.0	Runtime
torch	2.14.1+cu126	Tensor library with CUDA 12.6
torchvision	0.29.1+cu126	Required by transformers image processors and spandrel
diffusers	0.41.0	SDXL and FLUX pipelines, schedulers, LoRA loading
transformers	5.20.2	CLIP and T5 text encoders
accelerate	1.15.0	Device placement and CPU offload
safetensors	0.8.0	Safe weight loading
huggingface_hub	1.33.0	Model download and cache
peft	0.21.2	LoRA adapters on the UNet and text encoders
compel	2.5.1	Prompt weighting and long prompts for SDXL
bitsandbytes	0.50.2	4-bit and 8-bit quantisation for FLUX
sentencepiece, protobuf	0.2.2, 7.36.2	T5 tokenizer used by FLUX
gradio	6.30.0	Web interface, theme, timer
spandrel	0.4.2	Loads the Real-ESRGAN architecture and weights
pandas	3.0.6	History table in the UI
psutil	7.2.2	CPU, memory, disk and network statistics for the Admin panel
Pillow	12.3.0	Image I/O, resizing, cropping, figure assembly
numpy	2.5.4	Tile arrays for the upscaler
reportlab	5.0.1	Builds this documentation (development only)
NVIDIA driver	610.88	Also provides nvidia-smi for telemetry

2.5 Model assets

Asset	Source	Size	Location
stabilityai/stable-diffusion-xl-base-1.0 (fp16)	Hugging Face Hub	6.7 GB	HF cache
RunDiffusion/Juggernaut-XL-v9 (fp16)	Hugging Face Hub	6.5 GB	HF cache
SG161222/RealVisXL_V5.0 (fp16)	Hugging Face Hub	6.5 GB	HF cache
Lykon/dreamshaper-xl-v2-turbo (fp16)	Hugging Face Hub	6.5 GB	HF cache
playgroundai/playground-v2.5-1024px-aesthetic (fp16)	Hugging Face Hub	6.5 GB	HF cache
cagliostrolab/animagine-xl-3.1	Hugging Face Hub	6.5 GB	HF cache
Tongyi-MAI/Z-Image-Turbo (bf16)	Hugging Face Hub	20 GB	HF cache (+12 GB bf16 copy in models/cache/)
Qwen/Qwen-Image transformer, VAE, tokenizer	Hugging Face Hub	41 GB	HF cache (+ ~11 GB 4-bit copy in models/cache/)

Asset	Source	Size	Location
diffusers/qwen-image-nf4 text encoder	Hugging Face Hub	5.8 GB	HF cache
lightx2v Qwen-Image-Lightning 8-step V2.0 LoRA	Hugging Face Hub	0.8 GB	HF cache
black-forest-labs/FLUX.1-schnell	Hugging Face Hub (gated, login required)	33 GB	HF cache, not yet downloaded
RealESRGAN_x4plus.pth	github.com/xinntao/Real-ESRGAN releases v0.1.0	64 MB	models/ inside the project
User checkpoints and LoRAs	Civitai or own training	varies	models/checkpoints/, models/loras/

The Hugging Face cache is `%USERPROFILE%\.cache\huggingface\hub` on Windows and the `hf-cache` volume inside Docker.

2.6 Constraints and assumptions

- SDXL-family models are trained at about 1 megapixel. Native generation is therefore capped at a 1536 px long side; larger sizes are reached by upscaling.
- Only one model is resident at a time. Switching takes about 26 s for an SDXL-family model and longer for Z-Image Turbo; the first use of a model downloads it.
- Z-Image Turbo and FLUX.1 schnell have no negative prompt and no guidance scale, and LoRAs and prompt weighting are not applied to them. FLUX also requires a one-time Hugging Face login because the repository is gated.
- The telemetry module depends on nvidia-smi being on the PATH, which the NVIDIA driver installer guarantees on Windows. If it is missing, telemetry fields are stored as NULL and generation still works.
- The application is single-user and single-GPU. Concurrent requests are queued by Gradio and processed one at a time.
- Each model's licence (see Appendix E) permits local and commercial use with its own use restrictions; the operator is responsible for the content they generate.

3. System Architecture

3.1 Component overview

The application is a single Python process. Gradio serves the browser interface and calls plain Python functions; those functions use sixteen project modules, one resident image model, at most one resident video or music model, and the upscaler.

Component	File	Responsibility
Web application	app.py	Builds the Studio and History tabs, owns the ModelManager and the Upscaler, orchestrates one generation end to end, renders the details report, mounts the Admin panel.
Admin panel	admin_ui.py	Eight Admin sub-tabs and their handlers: models, LoRAs, add, downloads, system, settings, logs.
Model registry	models.py	Built-in specs, local checkpoints and admin-added models merged with registry overrides; ModelManager (load, swap, free VRAM, lock), samplers, family-aware LoRA activation, compel prompt encoding, quantised loaders for Z-Image, Qwen-Image and FLUX.
Persistent registry	registry.py	models/registry.json: settings, custom models and overrides, LoRA metadata (family, trigger words, weight, source, enabled).
Downloads	downloads.py	Source inspection (Hugging Face repo or file, Civitai, URL, local path), the background JobManager with progress, resumable file downloads, repo snapshots with byte progress, deletion of model files.
Theme and shared UI	ui_theme.py, ui_common.py	Colours, Gradio theme and CSS; dropdown choices, header, model cards, tiles, key-value tables and the downloads list.
Log buffer	applog.py	Tees stdout and stderr into a ring buffer for the Logs tab.
Video and music engine	media.py	Profiles for LTX-Video, Wan 2.1, MusicGen and Stable Audio; MediaManager (load straight to GPU, bf16 text-encoder cache, unload, generate_video, generate_music); prepare_for() GPU coordination with the image ModelManager.
Video and Music tabs	media_ui.py	Handlers and layout of both tabs: generation, frame-wise upscaling, soundtrack muxing, model defaults, chips, examples, media details report.
Video and audio files	video_io.py	ffmpeg (from imageio-ffmpeg) wrappers: streamed MP4 encoding, frame reading, first-frame thumbnails, audio muxing; WAV writing and waveform pictures.
Prompt parser	prompt_parser.py	Turns free text into a Settings object: model, LoRAs, size, steps, guidance, count, seed, style, upscale target. Pure function, no GPU.
Upscaler	upscaler.py	Wraps Real-ESRGAN through spandrel; tiled x4 passes; resizes or crops to the requested target.
GPU telemetry	gpu_info.py	Static GPU facts from torch; live temperature, power, clocks, fan and load from nvidia-smi.
System telemetry	sysinfo.py	CPU, memory, disks, network, GPU extras, power, software and process facts for the Admin panel, built on psutil and nvidia-smi.
Database	db.py	SQLite schema, migration, insert, update, fetch, search, per-row get, aggregate stats, command-line viewer.
CLI generator	generate.py	Standalone batch generation with any model and LoRA, without the web UI.

3.2 Generation pipeline

For every request the sequence is:

1 The prompt text is parsed into a Settings object. Model and LoRA tags and numeric directives are stripped; style words add descriptor suffixes.

2 The model is chosen: named in the prompt, selected in the Model box, or picked automatically from the style. The model's own step and guidance defaults are applied unless the prompt set them.

3 If the chosen model is not the resident one, the ModelManager frees the current pipeline and loads the new one with its recommended sampler.

4 Requested LoRAs are loaded (once) and activated with their weights; with none requested, adapters are disabled.

5 The prompt and negative prompt are encoded with compel into weighted embeddings (SDXL family) so that (word:1.3) and long prompts work; FLUX receives plain text.

6 For each image: the seed is fixed, peak-memory counters are reset and a telemetry snapshot is taken.

7 The pipeline runs for the requested number of steps at the native size. A per-step callback drives the progress bar.

8 CUDA is synchronised, the inference time is recorded, and a second telemetry snapshot is taken while the GPU is still under load.

9 The native image is saved as PNG. If an upscale target is set, Real-ESRGAN runs and the upscaled PNG is saved.

10 A record with about 50 fields is inserted into generations.db; the gallery, details report, Upscale bar and "Loaded now" line are updated in the browser.

A later click on the Upscale bar reads the native PNG of each selected record, upscales it, saves the result and writes the eight upscale columns onto the same row with db.update().

3.3 Data flow and storage

Data	Where it lives	Format	Written by
Native images	outputs/YYYYMMDD-HHMMSS_seedN.png	PNG, 1 to 2 MB	app.py, generate.py
Upscaled images	outputs/..._seedN_3840px.png	PNG, 8 to 31 MB	app.py
Generation records	generations.db	SQLite, one table	db.py
Built-in model weights	Hugging Face cache	safetensors	downloaded once
User checkpoints and LoRAs	models/checkpoints/, models/loras/	safetensors	the user
Upscaler weights	models/RealESRGAN_x4plus.pth	pth	downloaded once
Logs	stdout of the app process	text	app.py

3.4 Design decisions

Prompt as the only control surface. Early versions had dropdowns for style and size and sliders for steps and guidance. These were removed in favour of a parser because a single text box is faster to use, is self-documenting through the live interpretation line, and makes every generation reproducible from the prompt string alone. The Model box is the one exception, and even it can be overridden from the prompt.

One resident model, swapped on demand. Holding every model in VRAM would need 40 GB. The ModelManager keeps exactly one pipeline, swaps in about 26 s, and verifies the old one is freed. Automatic selection only switches to models that are already on disk so a style word never triggers a surprise 6.5 GB download.

Per-model defaults and samplers. A turbo model wants 6 steps at guidance 2, FLUX wants 4 steps at 0, SDXL wants 30 at 7. The parser records whether the prompt set steps or guidance explicitly; otherwise the model supplies them, and quality words scale them proportionally. Every SDXL-family model runs with the sampler its author recommends instead of the stock Euler scheduler.

compel for prompt weighting. The CLIP encoders truncate at 77 tokens and have no notion of emphasis. compel chunks long prompts and applies (word:1.3) weights to the embeddings. Its own padding helper does not support SDXL's dual encoder, so models.py pads mismatched chunk counts with the embedding of the empty prompt.

Two-stage resolution. SDXL produces incoherent output above roughly 1 megapixel. Generating at native size and upscaling with a super-resolution network is the standard approach and completes in a few seconds. The Upscale bar makes the second stage available after the fact, on the same database row.

Tiled upscaling. Real-ESRGAN on a full 3840 px image would need tens of gigabytes for intermediate feature maps. Processing 512 px tiles with a 32 px overlap bounds memory and hides seams.

Telemetry via nvidia-smi and psutil. Shelling out to nvidia-smi avoids a dependency on NVML bindings and costs about 100 ms per call. psutil covers CPU, memory, disks and network portably, so the Admin panel works unchanged inside the Docker image.

SQLite over a server database. A single-user desktop tool needs no server. The file is trivially backed up and any SQL client can open it. connect() adds missing columns so the schema can grow without migration scripts.

Always-dark theme. The Frazile AI palette is applied to both of Gradio's colour modes so the studio looks the same regardless of the browser's preference.

3.5 Concurrency model

Gradio runs the generate and upscale functions in a worker thread and queues additional requests. The GPU is used by one job at a time (a lock in ModelManager and the "gpu" queue id enforce it, while the statistics handlers run on their own queue). The History and Admin tabs read the database and the system on their own threads; SQLite handles the concurrent reads safely because each call opens its own short-lived connection, and the Admin collectors never touch the pipeline.

4. Workflows

4.1 First-time setup

1. Confirm the GPU and driver are visible: run nvidia-smi in a terminal. It must list the card and a driver version.
2. Open PowerShell in the project folder.
3. Create and activate a virtual environment: `python -m venv .venv` then `.\.venv\Scripts\Activate.ps1`
4. Install the CUDA build of PyTorch, then the remaining dependencies: `pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126` and `pip install -r requirements.txt`
5. Verify CUDA is active: `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"`
6. Confirm the upscaler weights exist at models\RealESRGAN_x4plus.pth. If not, download them from the Real-ESRGAN v0.1.0 release page into that folder.
7. Start the app once. The first start downloads the 20 GB Z-Image Turbo model into the Hugging Face cache. Other models download the first time they are used.

4.2 Daily use: generating an image

1. Double-click run-app.bat, or run `.\.venv\Scripts\python.exe app.py` from the project folder.
2. Wait for "Z-Image Turbo ready" and "Upscaler ready" in the console (about a minute). The browser opens http://127.0.0.1:7860 automatically.
3. Type a prompt. As you type, the line under the box shows the interpreted settings and the model, for example "1344x768 | upscale to 3840x2194 | 40 steps | guidance 5 · model Juggernaut XL v9".
4. Optionally pick a model in the Model box (Auto is the default) and edit the negative prompt.
5. Press Generate or Enter. A progress bar shows the model being loaded if it changed, then the step count and, if applicable, the upscale stage.
6. The image appears in the gallery. The Upscale bar appears under it, and the details panel shows timing, settings, GPU telemetry and the exact prompt sent to the model.
7. Both files are already saved in outputs\. Clear resets the prompt and results.



Figure 4.1: "a lighthouse on a rocky cliff at golden hour, cinematic, 16:9, 4k, seed 42" rendered at 1344x768 and upscaled to 3840x2194.

4.3 Choosing a model

- Say nothing: Auto keeps Z-Image Turbo (loaded at start) or FLUX. When an SDXL-family model is loaded it switches to the specialist for a style word if that model is on disk (photorealistic to Juggernaut, anime to Animagine, oil painting or watercolor to Playground, digital art, comic or cyberpunk to DreamShaper).
- Name it in the prompt: `model juggernaut`, `model: flux`, `checkpoint=realvis`, `using dreamshaper`, `with playground`.
- Pick it in the Model box. A prompt directive always wins over the box.
- The Models accordion under the prompt lists every model, its default steps and guidance, whether it is on disk, and what it is best for.
- Steps and guidance follow the model. Quality words still scale them: `quick` halves the steps, `highly detailed` adds 35 %, `strict` and `creative` move guidance by 30 %.

4.4 Using LoRAs and your own checkpoints

- Copy an SDXL LoRA file to models\loras\, for example pixel-style.safetensors, and write `<lora:pixel-style:0.8>` anywhere in the prompt. Several tags can be stacked; the weight is optional and defaults to 1.
- Copy an SDXL checkpoint (a single .safetensors file, as distributed on Civitai) to models\checkpoints\ and restart the app. It appears in the Model box and can be named in the prompt by its file name. Files with turbo, lightning or hyper in the name get 6-step defaults.
- Two LoRAs are included: pixel-art-xl and papercut. Tick them in the LoRAs box or tag them in the prompt. SDXL LoRAs apply to SDXL-family models; a LoRA that cannot be found or does not fit the loaded model is reported in the notes under the details report rather than failing the generation.

4.5 Producing 4K and 8K output

From the prompt: include 4k, 8k, 2k or an exact size such as 3840x2160. The parser sets an upscale target and raises steps by a third for a cleaner base. Generation runs at the native bucket for the aspect ratio, then the upscaler runs one x4 pass for 4K or two passes for 8K, and an exact size is centre-cropped.

After generation: the Upscale bar under the gallery offers 2K, 4K and 8K. Click an image to upscale only that one, or leave nothing selected to upscale the whole batch. The gallery switches to the upscaled files and the history rows gain the upscale columns. The native PNG is always kept.

Recommended practice: iterate at native size with a fast model until the composition is right, then upscale the chosen image from the bar. That costs 3 to 9 seconds instead of a regeneration.



Figure 4.2: The same 300x225 region of the lighthouse cliff. Left: native pixels enlarged. Right: Real-ESRGAN output. The upscaler reconstructs rock texture rather than blurring.

4.6 Reviewing past generations

1. Open the History tab. The top line shows totals: number of images, GPU seconds, average and extreme timings, VRAM, hottest temperature and peak power.
2. Type part of a prompt in the search box and press Enter to filter, or press Refresh to reload. The table now has a model column.
3. Click a row. The image (upscaled if one exists) and its full details appear below the table.
4. For scripted access, run `db.py`, `db.py 12` or `db.py stats` with the virtual environment's Python.

4.7 Adding a model or a LoRA

1. Open Admin, then Add. Paste a source: a Hugging Face repo id such as RunDiffusion/Juggernaut-XL-v9, a file in a repo such as nerijs/pixel-art-xl/pixel-art-xl.safetensors, a Civitai model link, a direct https link to a .safetensors file, or a local path. Press Check.
2. The result line says what was found: pipeline class and family for a diffusers repo, file name and size for a file, type, base model and trigger words for a Civitai entry, and warnings (gated repo, token needed, several files). The form is pre-filled: Add as (model, checkpoint file or LoRA), key, name, family, trigger words, default steps, guidance and sampler.
3. Press Add. A diffusers repo is registered at once and downloaded in the background if Download now is ticked; a file is downloaded into models/checkpoints/ or models/loras/ and registered when it finishes. The Downloads tab shows bytes, speed and remaining time; the header shows the number of running downloads.
4. The new model appears in the Model box (and answers to "model <key>" in the prompt) and the new LoRA in the LoRAs box without a restart. Small files can also be uploaded from the browser at the bottom of the Add tab.
5. To remove something: Models tab, select it, Remove from list (a built-in is reset to its defaults instead) or tick the confirmation and Delete files from disk; LoRAs tab, select it, tick the confirmation and Delete LoRA file.

4.8 Managing models and LoRAs

1. Models tab: click a row. Edit name, description, default steps and guidance, sampler, weight variant, aliases, auto-pick styles, negative-prompt support and Enabled, then Save changes. Load now loads it immediately

(with progress); Download in background fetches it without loading; Set as start-up model changes what loads when the app starts; Unload current model frees the VRAM.

2 LoRAs tab: click a row. Set the family it was trained for (SDXL, Z-Image, Qwen-Image or FLUX), its trigger words, default weight, description, source and tags, then Save. Try it in the Studio fills a test prompt with the trigger words, the LoRA tag and a matching model and switches to the Studio tab.

3 Settings tab: start-up model, the weight used for LoRAs ticked in the LoRAs box when they have no weight of their own, the default negative prompt, the Hugging Face token (required for FLUX and gated repos; Check login verifies it), the Civitai token, clean-up of generations older than N days, and the admin password lock.

4.9 Watching the machine: Overview, System and Logs

1 Admin, Overview: system tiles, activity tiles (images today and this week, GPU minutes in 24 h, loaded and start-up model, models on disk, LoRAs, downloads, outputs size, Hugging Face login) and the per-model statistics table.

2 Admin, System: the nine tiles and six detail sections of section 9.4. Press Refresh for a new sample, or tick Auto-refresh every 5 s while a generation runs to watch VRAM, temperature and power move; statistics keep refreshing during generation.

3 Tick "Check internet reachability" to add a TCP connect test to Hugging Face, GitHub and PyPI; it is off by default because an offline machine would wait 2 s per host.

4 Admin, Logs: the last lines the app printed (model loads, generations, LoRA notes, downloads, errors) with a text filter, an errors-only level and auto-refresh.

4.10 Batch generation from the command line

```
.\.venv\Scripts\python.exe generate.py --list-models
.\.venv\Scripts\python.exe generate.py "a red sports car" --model juggernaut --seed 42 --count 4
.\.venv\Scripts\python.exe generate.py "cat astronaut" --model dreamshaper --lora pixel-style:0.7
.\.venv\Scripts\python.exe generate.py --help
```

The script loads the model per run, so it is best for batches. It writes to outputs\ but does not record to the database or upscale.

To generate a whole category with the app's own pipeline (database record, telemetry, category folder and prompt library), keep the app running and use batch.py:

```
.\.venv\Scripts\python.exe batch.py --category Nature --count 50
.\.venv\Scripts\python.exe batch.py --category Cities --prompts my_prompts.txt --model juggernaut
 --seed-start 100
.\.venv\Scripts\python.exe db.py prompts Nature
```

Prompts come from a built-in set (Nature: 50 prompts covering mountains, forests, oceans, deserts, wildlife, macro, weather and seasons) or a text file with one prompt per line; they are cycled with fresh seeds when the count is larger than the set, and --skip resumes an interrupted batch at a given prompt. Each image is saved to outputs/<category>/ with its category in generations and its prompt in prompt_library.

4.9 Upscaling an existing image

```
.\.venv\Scripts\python.exe upscaler.py outputs\some_image.png 3840
```

The result is written next to the input with an _up3840 suffix. For images made in the app, prefer the Upscale bar so the history row is updated too.

4.10 Enabling FLUX.1

1 Accept the licence at https://huggingface.co/black-forest-labs/FLUX.1-schnell while logged in to Hugging Face.

2 Run `.\.venv\Scripts\huggingface-cli login` and paste a read token.

3 Say `model flux` in a prompt. The first use downloads about 33 GB; on a 24 GB card the transformer is quantised to 4-bit and the T5 encoder to 8-bit at load time, which takes a few minutes.

4 FLUX ignores the negative prompt, guidance words and LoRAs. Four steps are enough.

4.11 Running in Docker

1 Install the prerequisites from section 11 (on Windows: WSL 2 and Docker Desktop).

2 In the project folder run `docker compose up --build`. The first build installs PyTorch and the dependencies (several GB).

3 Open http://localhost:7860. Images land in outputs\, the database is generations.db, and downloaded models persist in the hf-cache volume.

4 Stop with Ctrl+C or `docker compose down`. The volume survives; `docker compose down -v` deletes the cached models.

4.12 Stopping, restarting and backup

- Close the console window, or press Ctrl+C in it, to stop the server. Loaded models are released.
- Restart with run-app.bat. Startup is about 30 to 45 seconds because the model is reloaded from disk.
- If port 7860 is reported busy, a previous instance is still running; end it from Task Manager (python.exe) or set GRADIO_SERVER_PORT.
- Back up the outputs\ folder, generations.db and models\ together. Built-in models can be re-downloaded and the virtual environment rebuilt from requirements.txt.

4.14 Generating a video clip

1 Open the Video tab. Describe the scene, the motion and the camera in one or two sentences; the chips add camera and motion hints (tracking shot, drone shot, slow motion, golden hour ...).

2 Pick the model (LTX-Video 2B by default; Wan 2.1 for smoother but slower motion), a size, a length in seconds, and optionally steps, guidance and a seed. The model box resets steps, guidance and the negative prompt to the model's defaults.

3 Optional: drop a start image into the Start image box (LTX only). It is centre-cropped to the chosen size and animated (image-to-video).

4 Press Generate video. The first use unloads the image model, loads the video model (about 2 minutes the first time, about a minute afterwards) and shows step progress; the clip plays in the result box and its details appear below.

5 Optional: Upscale video (Full HD, 2K or 4K) runs every frame through Real-ESRGAN and re-encodes the clip; Add soundtrack lays a track from the Music tab under it. Both produce new files next to the original.

6 Clips are in outputs/ (<timestamp>_seed<seed>.mp4, _up<px>.mp4, _music.mp4) and in History under kind = video.

4.15 Generating a music track

1 Open the Music tab. Describe genre, mood, instruments and tempo; the chips add common genres and moods.

2 Pick the model (MusicGen stereo medium by default), the length (3 to 30 s), guidance, temperature and optionally a seed.

3 Press Generate music. MusicGen loads in about 30 s (it can stay loaded next to an SDXL model) and writes the track with a progress bar; the player appears with the details below.

4 Tracks are 16-bit WAV files in outputs/ and appear in History under kind = music with a waveform thumbnail. The Video tab lists them in its Soundtrack box.

4.13 Development workflow

1. Edit a module. Self-tests: `python prompt_parser.py` prints how sample prompts are interpreted, `python models.py` lists the registry and what is on disk, `python registry.py` prints the registry file, `python downloads.py <source>` inspects a source, `python sysinfo.py` dumps the Admin data as JSON, `python db.py stats` exercises the schema.
2. Compile-check with `python -m py_compile app.py admin_ui.py media_ui.py media.py video_io.py models.py registry.py downloads.py prompt_parser.py sysinfo.py`. `python media.py` lists the video and music models and whether they are on disk.
3. Restart the app. For automated checks, gradio_client can call /generate, /upscale, /refresh_history, /overview_refresh, /system_refresh, /check_source, /add_source, /save_lora and /logs_text, which is how the increments in this project were verified.
4. Rebuild this documentation with `python docs-src\build_pdf.py`.
5. Commit and push: the repository is https://github.com/Fzhacking/ai-image-gen.

5. Module Reference

5.1 app.py (about 640 lines)

Entry point of the web application: the Studio and History tabs, generation, and the mounting of the Admin panel.

Name	Kind	Description
OUTPUT_DIR, UPSCALE_CHOICES, HELP, EMPTY_STATE, THUMB_DIR	constants	Outputs path, the 2K/4K/8K choices, the prompt help table, the empty-gallery message and the thumbnail cache folder.
MANAGER, STARTUP_MODEL, UPSCALER, STATIC_GPU	globals	The ModelManager, the start-up model (FRAZILE_STARTUP_MODEL, else the registry setting, "none" to skip), the Upscaler and static GPU facts, created once at import. applog.install() runs first so every later print reaches the Logs tab.
ui_lora_weight(stem), default_negative()	functions	Registry-backed defaults: a LoRA's own weight (else the global setting) and the default negative prompt.
pick_model(settings, choice)	function	Resolves the model: prompt directive, then dropdown, then (Auto) a model of the family the requested LoRAs were trained for, then style-based choice among SDXL specialists. Returns (spec, note).
add_ui_loras(), trigger_hints(), preview()	functions	Merge ticked LoRAs into the settings, remind of missing trigger words, and render the live "Will generate" line (including whether a load or download is coming).
details_markdown(rec), upscale_record(rec, target_px, exact_size)	functions	The Markdown report for one record; upscaling of a record's native PNG.
generate(prompt, negative, model_choice, loras_ui, progress)	function	Runs the full pipeline under the ModelManager lock for all requested images; returns gallery, report, records, selection, Upscale bar, hint, loaded line and empty-state visibility.
clear_all(), upscale_hint(), on_select(), upscale()	functions	Clear button, the hint under the Upscale bar, gallery selection tracking, and the Upscale bar handler.
toggle_chip(), chip_seed(), reuse_seed(), drop_seed()	functions	Quick-hint chips (a hint in the same group replaces its siblings), the random-seed chip, Reuse this seed and New variation.
thumbnail_for(), history_rows(), history_table(), history_gallery(), history_stats(), refresh_history(), show_record(), select_history(), select_history_thumb(), reuse_prompt(), delete_history()	functions	History tab: 384 px JPEG thumbnails cached in outputs/.thumbs/, filters (search, model, upscaled only, limit), table and grid views, details, reuse prompt with or without seed (switches to the Studio tab) and deletion.
refresh_header()	function	The header with loaded model, VRAM, temperature, GPU load and running downloads, refreshed by a 5 s timer.

The UI is a gr.Blocks with fill_width=True and three tabs (Studio, History, Admin). GPU events (generate, upscale, admin load and unload) share the concurrency id "gpu" so they queue behind each other; statistics and timers use "stats" with a limit of three so they keep refreshing during a generation. launch() reads GRADIO_SERVER_NAME, GRADIO_SERVER_PORT and FRAZILE_NO_BROWSER, allows 8 GB uploads and passes the theme and CSS.

5.2 models.py (about 680 lines)

Name	Kind	Description
ModelSpec	dataclass	key, name, source (HF repo or path), family (sdxl, zimage, qwen or flux), steps, guidance, scheduler, variant, negative, aliases, styles, description, download_gb, local.
BUILTIN	constant	The nine built-in specs with their defaults and recommended samplers.

Name	Kind	Description
local_checkpoints(), all_models(include_disabled), family_models(family), model_words()	functions	Scan models/checkpoints/; the merged list (built-ins with registry overrides, local checkpoints, admin-added models, minus disabled ones); the on-disk models of one family in preferred order; every key and alias for the parser.
resolve(name)	function	Finds a model by key, alias, name, repo id or path, including disabled ones; raises KeyError if unknown.
model_for_style(style), is_downloaded(spec), on_disk_bytes(spec), hf_cache_dir(repo), display_name(source), spec_for_source(source)	functions	Auto-selection lookup, on-disk check and size (file, HF cache blobs, converted caches), cache folder of a repo, and source-to-spec mapping for the UI.
find_lora(name), available_loras(family), lora_info(name)	functions	Lookup by file stem or registry display name; the enabled LoRA stems, optionally of one family; a LoRA's registry metadata.
a1111_to_compel(prompt)	function	Rewrites (word:1.3) into compel's (word)1.3.
ModelManager	class	Holds one pipeline and a re-entrant lock. load() dispatches to the SDXL, Z-Image, Qwen-Image or FLUX loader; unload() frees VRAM; apply_loras() skips LoRAs of another family, loads once and activates with weights on top of base adapters; encode() returns weighted embeddings via compel; release_cache(); generation_kwargs() adds family-specific arguments.
STARTER_LORAS, fetch_starter_loras()	constant, function	The two documented LoRAs and the `models.py fetch-loras` command.

Running the module directly lists every model, its family and whether it is downloaded.

5.3 registry.py (about 300 lines)

Name	Kind	Description
REGISTRY_PATH, FAMILIES, SCHEDULERS, FAMILY_PREFERRED, DEFAULT_SETTINGS, OVERRIDABLE	constants	models/registry.json; the four families with descriptions; sampler names; which models to prefer per family; default settings (startup_model, ui_lora_weight, default_negative, civitai_token, auto_release_vram, outputs_keep_days); the ModelSpec fields an admin may override on a built-in.
load(), save()	functions	Cached read (re-read when the file changes) and atomic write with an updated_at stamp.
setting(key), set_settings(key=value ...)	functions	Settings access.
apply_overrides(spec), custom_models(), model_entry(), is_enabled(), is_custom(), upsert_model(key, custom, key=value ...), remove_model(key), set_enabled()	functions	Model entries: overrides for built-ins and local files, full specs for admin-added models, enable flag, create or update, delete or reset.
lora_entries(include_missing), lora(stem), set_lora(stem, key=value ...), forget_lora(stem)	functions	LoRA records: files in models/loras/ merged with their metadata (name, family, trigger, weight, description, source, enabled, tags, size, missing flag).
summary()	function	Counts for the Admin Overview.

5.4 downloads.py (about 480 lines)

Name	Kind	Description
Job, JobManager, JOBS	dataclass, class, global	A download with status, bytes done and total, speed, ETA, message, log and a cancel flag; a two-worker thread pool that runs job functions and keeps the list; the shared instance.

Name	Kind	Description
inspect_source(text)	function	Classifies a source as hf_repo, hf_file, civitai, url or local and gathers what it would give: pipeline class and family (from model_index.json), fp16 variant, sizes, gated flag, Civitai type, base model and trigger words, warnings. Never raises.
download_file(info, as_type, rename)	function	Job function: streams one .safetensors file into models/checkpoints/ or models/loras/ with resume (Range), progress, Hugging Face and Civitai tokens, and refuses HTML answers.
download_repo(spec)	function	Job function: snapshot_download of a diffusers repo with ignore patterns per family (fp16 only for SDXL, no bin or onnx, no Qwen text encoder) and byte progress read from the cache folder; fetches the Qwen 4-bit text encoder and Lightning LoRA too.
delete_model_files(spec)	function	Removes a local file or the repo's HF cache folder and the converted caches under models/cache/.
hf_token(), civitai_token(), safe_stem(), unique_path(), human_bytes()	functions	Helpers.

Running `python downloads.py <source>` prints the inspection result as JSON.

5.5 admin_ui.py (about 600 lines)

Name	Kind	Description
build_admin(manager, upscaler, static_gpu, prompt_box, tabs, model_choice, lora_choice, loaded_md)	function	Builds the eight sub-tabs inside the Admin tab and wires their events; receives the Studio's prompt box, tab bar and dropdowns so Admin actions can update them.
guard(unlocked, confirm), unlock()	functions	Confirmation box and optional FRAZILE_ADMIN_PASSWORD check for destructive actions.
overview_refresh()	function	System tiles, activity tiles and the per-model statistics table.
models_df(), select_model(), model_editor_values(), save_model(), load_model(), unload_model(), download_model(), set_startup(), reset_or_remove_model(), delete_model_files()	functions	Models tab.
loras_df(), select_lora(), lora_editor_values(), save_lora(), try_lora(), delete_lora()	functions	LoRAs tab; try_lora fills a test prompt and switches to the Studio.
check_source(), add_source(), upload_file()	functions	Add tab: inspection with form pre-fill, registration plus background download, browser upload.
downloads_refresh(), cancel_job(), clear_jobs()	functions	Downloads tab.
system_refresh(check_internet)	function	The nine tiles and six sections of the System tab.
settings_values(), save_settings(), hf_status_html(), save_hf_token(), cleanup_outputs()	functions	Settings tab.
logs_text(n, contains, level)	function	Logs tab.

5.6 ui_theme.py, ui_common.py, applog.py

Module	Contents
ui_theme.py (about 190 lines)	Colour constants, build_theme() (Frazile AI palette applied to light and dark mode alike), CSS for header, chips, cards, badges, tiles, meters, jobs and logs, and the badge helpers.
ui_common.py (about 170 lines)	model_choices() and lora_choices() for the Studio dropdowns with update helpers; header_html() with live pills; loaded_line(); model_cards_html(); tile(), kv() and jobs_html().
applog.py (about 120 lines)	install() tees stdout and stderr into a 3,000-line ring buffer (keeping only the final state of progress bars; lines mentioning error, exception, traceback or failed count as errors); log(), tail(), tail_text(), counts().

5.7 prompt_parser.py (315 lines)

Name	Kind	Description
Settings	dataclass	prompt, width, height, steps, guidance, count, seed, style, upscale_to, exact_size, notes, model, loras, steps_explicit, guidance_explicit, quality, fidelity. Methods: summary(), upscaled_size(), apply_defaults(base_steps, base_guidance).
RATIO_BUCKETS, ORIENTATION_WORDS, STYLE_SUFFIXES, UPSCALE_TARGETS	constants	As in v1.0: eleven SDXL buckets, orientation regexes, twelve style presets, upscale words.
parse(text)	function	Returns a Settings. Order: LoRA tags, model directive, explicit resolution, ratio, orientation, upscale words, steps, guidance, count, seed, style, tidy-up, then apply_defaults with the SDXL baseline.

Quality and fidelity words no longer set fixed numbers; they are recorded and applied relative to the chosen model's defaults. Running the module directly prints the interpretation of twelve sample prompts.

5.8 sysinfo.py (358 lines)

Name	Kind	Description
human_bytes(), human_duration()	functions	Formatting helpers used by the Admin panel.
cpu()	function	Processor name (registry on Windows, /proc/cpuinfo on Linux), cores, frequency, usage per core, load averages, process and thread counts (cached 30 s), boot time, uptime.
memory(), disks()	functions	RAM and swap; partitions with usage, disk I/O totals, and the sizes of outputs/, models/, the HF cache and the database (cached 60 s).
network(), internet()	functions	Hostname, primary IP, totals, per-second rates from the previous sample, errors and drops, connections and listening ports, every interface; optional TCP reachability of three hosts.
gpu(), power()	functions	gpu_info static and live data plus PCIe link, VBIOS, max clocks, power limits, throttle reasons and GPU processes from nvidia-smi; power draw against limit, Windows power plan, battery.
software(), app(), everything()	functions	Library and driver versions and the Docker flag; the app process; one call that collects all of the above.

5.9 upscaler.py (109 lines)

Name	Kind	Description
MODEL_PATH, TARGETS, TILE, OVERLAP	constants	models/RealESRGAN_x4plus.pth; 2k 2560, 4k 3840, uhd 3840, 8k 7680; 512 and 32 pixels.
Upscaler(device, model_path)	class	Loads the model with spandrel, switches it to half precision on CUDA, exposes name and scale.
Upscaler.upscale_once(img)	method	One x4 pass over padded tiles under inference_mode.

Name	Kind	Description
Upscaler.upscale_to(img, long_side)	method	Repeats passes until the long side reaches the target, resizing before a second pass, then Lanczos-resamples to exactly long_side. Returns (image, info).

5.10 gpu_info.py (97 lines)

Name	Kind	Description
snapshot()	function	Runs nvidia-smi once and returns temperature_c, power_w, power_limit_w, gpu_util_pct, mem_util_pct, mem_used_mb, mem_total_mb, sm_clock_mhz, mem_clock_mhz, fan_pct, driver_version, pstate. Returns {} if nvidia-smi fails.
static_info()	function	gpu_name, vram_total_gb, compute_capability, multiprocessors, cuda_version, cudnn_version, torch_version, diffusers_version, python_version, os.
torch_memory()	function	vram_allocated_gb, vram_reserved_gb, vram_peak_gb from the torch allocator.

5.11 db.py (about 270 lines)

Name	Kind	Description
DB_PATH, SCHEMA, NEW_COLUMNS	constants	generations.db next to the module; CREATE TABLE plus index; columns added after the first release.
connect()	function	Opens a connection with Row factory, applies schema and migrations.
insert(record)	function	Inserts known keys as columns; unknown keys are packed into extra_json. Returns the row id.
update(row_id, fields)	function	New in v2.0: sets known columns on one row; used by the Upscale bar.
fetch(limit, search), fetch_filtered(limit, search, model_id, upscaled_only), get(row_id), stats()	functions	Newest-first list with LIKE search, the filtered variant used by History, one record, aggregates.
delete(row_id, remove_files), stats_by_model(), recent_counts(), lora_usage(), models_used()	functions	New in v2.1: delete a record and optionally its files; per-model images, timings, VRAM and last use; images today, this week and GPU seconds in 24 h; LoRA use counts from the parser notes; distinct models for the History filter.

5.12 generate.py (125 lines)

Standalone command-line generator. Arguments: prompt, --negative, --steps, --guidance, --width, --height, --seed, --count, --model (key, repo id or path), --lora NAME[:WEIGHT] (repeatable), --out, --list-models. Uses the same ModelManager, so weighting, samplers and LoRAs behave exactly as in the app. It does not write to the database.

5.14 media.py (about 330 lines)

The video and music engine. PROFILES describes each architecture (ltx, wan, musicgen, stable_audio): kind, fps and frame-count rule, maximum length, default steps and guidance, negative prompt, VRAM need and whether image-to-video is supported. arch_of() reads the repo's model_index.json or config.json (cached) so admin-added repos get the right loader; profile_for() merges the registry's steps and guidance; repo_allow_patterns() limits a download to the diffusers sub-folders (the LTX repo carries 250 GB of single-file checkpoints at its root) or to the transformers files of MusicGen; frames_for() rounds a length in seconds to 8k+1 (LTX) or 4k+1 (Wan) frames.

- MediaManager(lock): one loaded video or music model. load() dispatches to _load_ltx, _load_wan, _load_musicgen or _load_stable_audio; each diffusers part is loaded with device_map onto the GPU shard by shard, because the fp32 text encoders (19 to 23 GB on disk) do not fit the machine's free RAM.

_text_encoder() converts to bf16 on the first load and saves a copy under models/cache/<key>-text_encoder-bf16/ for later loads, and can quantise to 8-bit with bitsandbytes (Wan on 24 GB cards).

- generate_video(prompt, negative, width, height, frames, steps, guidance, seed, image, on_step): returns PIL frames. LTX uses frame_rate 24, decode_timestep 0.03 and decode_noise_scale 0.025, VAE tiling, and an LTXImageToVideoPipeline built from the same components when a start image is given. Wan uses the UniPC scheduler with flow_shift 3.0 (480p) and an fp32 VAE.
- generate_music(prompt, negative, seconds, guidance, temperature, seed, on_progress): returns (samples, sample_rate). MusicGen runs in fp16 with do_sample, guidance_scale and max_new_tokens = seconds x 50; a StoppingCriteria reports token progress. Stable Audio calls the diffusers pipeline with audio_end_in_s.
- prepare_for(spec, image_manager, media_manager): the GPU rule. Loading an image model unloads a video model (and a music model if the image model is one of the big flow models); loading a video model unloads the image model; loading a music model unloads the image model only when free VRAM is below the music model's need plus 2 GB.

5.15 media_ui.py (about 420 lines)

- generate_video(): resolves the model, size, frame count and seed, crops the optional start image, takes the GPU lock, loads through prepare_for(), generates with step progress, encodes the MP4 with video_io.write_mp4, records the row (kind video, width, height, frames, fps, duration_s plus the usual telemetry) and returns the player value, the details report, a state dict and the upscale bar.
- upscale_video(state, target_px): reads the current clip frame by frame, runs the compact video upscaler (video_upscaler(): realesr-general-x4v3 through the same Upscaler class, one x4 pass plus a resample) on each frame and streams the result into a new MP4 (crf 18), keeping the sound track if one was added; updates the upscale columns of the record.
- add_soundtrack(state, track): muxes a track from the Music tab under the current clip with a one-second fade-out; stored in soundtrack_path.
- generate_music(): the same shape for tracks (kind music, duration_s, sample_rate), written as 16-bit WAV.
- video_defaults() / music_defaults(): fill steps, guidance, negative prompt, maximum length and the start-image box when the model box changes. build_video_tab() and build_music_tab() lay the tabs out and wire the events; refresh_*_choices() update the model and track boxes when a tab is opened.

5.16 video_io.py (about 190 lines)

- write_mp4(frames, path, fps, crf, audio): pipes raw RGB frames into the bundled ffmpeg (libx264, yuv420p, faststart) so a 4K clip is never held in memory; read_frames(path) yields PIL frames with the clip's metadata; probe(), video_thumbnail(), mux_audio().
- write_wav(samples, sample_rate, path) through soundfile (peak-normalised when needed); write_mp3(); waveform_png() draws the History thumbnail with the studio's pink-to-yellow gradient; audio_seconds().

5.13 Supporting files

File	Purpose
requirements.txt	Python dependencies with the note on installing the CUDA PyTorch build first
run-app.bat	Double-click launcher that runs app.py inside the virtual environment
batch.py	Category batch generation through the running app (built-in prompt sets or a prompt file)
README.md	User-facing quick start, model table, prompt reference, Admin and Docker sections
Dockerfile, docker-compose.yml, .dockerignore	Container image and service definition (section 12)

File	Purpose
models/RealESRGAN_x4plus.pth	Upscaler weights for images
models/realesr-general-x4v3.pth	Compact Real-ESRGAN weights for video frames (4.7 MB, fetched on the first video upscale)
models/checkpoints/, models/loras/	Checkpoints and LoRAs, filled by the Admin panel or by hand
models/registry.json	Admin registry: settings, model overrides and additions, LoRA metadata (created on first edit, not in git)
outputs/.thumbs/	Thumbnail cache for the History grid (JPEG for images and first frames of videos, PNG waveforms for tracks)
models/cache/<key>-text_encoder-bf16/	bf16 copies of the video models' text encoders, written on the first load (not in git)
docs/	This documentation set as PDFs
docs-src/build_pdf.py, docs-src/images/	Source of this documentation and its figures

6. Models and Prompt Enhancement

6.1 Built-in models

Every model is described by a ModelSpec in models.py: where it comes from, its family, its default steps and guidance, the sampler it runs with, the style words that auto-select it and a one-line description.

Say	Model	Family	Steps	CFG	Sampler	Best for
model zimage	Z-Image Turbo	zimage	8	0	flow matching (shipped)	default model: photoreal, text rendering and strong prompt following in 8 steps
model sdxl	SDXL base 1.0	sdxl	30	7	DPM++ 2M Karras	general purpose baseline
model juggernaut	Juggernaut XL v9	sdxl	30	5	DPM++ 2M Karras	photoreal people, products and scenes; the best all-round upgrade
model realvis	RealVisXL V5.0	sdxl	30	4	DPM++ SDE Karras	photoreal with strong skin, fabric and lighting detail
model dreamshaper	DreamShaper XL v2 Turbo	sdxl	6	2	DPM++ SDE Karras	fast illustration, fantasy and stylised art
model playground	Playground v2.5	sdxl	40	3	EDM (shipped)	highly aesthetic, painterly-real composition and colour
model animagine	Animagine XL 3.1	sdxl	28	6	Euler a	anime and manga characters
model qwen	Qwen-Image + Lightning	qwen	8	1	flow matching, exponential shift	20B model in 4-bit: best text rendering and layout, posters and signage
model flux	FLUX.1 schnell	flux	4	0	flow matching (shipped)	best prompt understanding: text, counting, complex scenes

Aliases also work: z-image, zit, jugg, realistic, turbo, aesthetic, anime-xl, qwen-image, schnell. Playground keeps its own EDM scheduler because its weights were trained with it; Z-Image, Qwen-Image and FLUX use flow-matching schedulers.



Figure 6.1: The same prompt ("close-up portrait of an old fisherman with weathered skin, golden hour light, photorealistic, seed 11") rendered by the six SDXL-family models at 832x1216 with their default settings.

6.2 Automatic selection

When the Model box is on Auto and the prompt names no model, the loaded model is kept while it is Z-Image Turbo or FLUX, because both handle every style themselves. When an SDXL-family model is loaded the style preset decides: photorealistic and cinematic choose Juggernaut, anime chooses Animagine, oil painting and watercolor choose Playground, digital art, comic and cyberpunk choose DreamShaper. The switch happens only if that model is already on disk; otherwise the loaded model is kept and the notes say which model to request. With no style word the loaded model is reused, so consecutive prompts never pay a swap unless they ask for one.

6.3 Steps and guidance follow the model

Prompt says	SDXL (30 / 7)	DreamShaper Turbo (6 / 2)	FLUX (4 / 0)
nothing	30 steps, cfg 7	6 steps, cfg 2	4 steps
quick	15, 7	3, 2	2
highly detailed or 4k	40, 7	8, 2	5
strict	30, 9.1	6, 2.6	4 (note: no guidance)

Prompt says	SDXL (30 / 7)	DreamShaper Turbo (6 / 2)	FLUX (4 / 0)
creative	30, 4.9	6, 1.4	4 (note: no guidance)
25 steps cfg 3	25, 3	25, 3	25, 3

6.4 Prompt weighting and long prompts

- `(glowing sword:1.3)` or `(glowing sword)++` emphasises; `(word:0.7)` or `(word)--` de-emphasises. Automatic1111-style syntax is converted to compel's.
- Prompts longer than 77 tokens are encoded in chunks instead of being cut off. Both positive and negative embeddings are padded to the same length with the empty-prompt embedding.
- Applies to every SDXL-family model, including local checkpoints. FLUX receives the plain text; its T5 encoder reads 256 tokens natively.
- If compel ever fails on a prompt, the pipeline's own encoder is used and a line is printed; generation never fails because of weighting.

6.5 LoRAs

Two starter SDXL LoRAs are set up by `python models.py fetch-loras` (they are not kept in git): **Pixel Art XL** (nerijs/pixel-art-xl, 163 MB, trigger word "pixel art") and **Papercut** (TheLastBen/Papercut_SDXL, 325 MB, trigger word "papercut"). Tick them in the LoRAs box under the negative prompt, where they apply at weight 0.8, or write the tag in the prompt with any weight.

- Files in models/loras/ are matched by name, ignoring case and punctuation: `<lora:Pixel_Art_XL:0.9>` finds pixel-art-xl.safetensors.
- LoRAs trained for SDXL apply to every SDXL-family model. Z-Image and FLUX accept LoRAs trained for them; an SDXL LoRA requested on those models is reported as incompatible in the notes and skipped.
- Each LoRA is loaded into the resident pipeline once and then activated or deactivated per request with its weight; swapping models reloads them on demand.
- LoRAs patch the UNet and the text encoders, so they interact correctly with weighting.
- Weights are clamped to the range -2 to 2.

6.6 Local checkpoints

Any single-file SDXL checkpoint in models/checkpoints/ is loaded with diffusers' from_single_file. The key is the file name in lower-case kebab form, the description shows its size, and turbo, lightning or hyper in the name selects 6 steps, guidance 2 and the SDE sampler. The folder is scanned at start-up.

6.7 Z-Image Turbo, the default model

Z-Image Turbo (Tongyi-MAI, Apache 2.0) is a 6-billion-parameter single-stream flow transformer with a Qwen3 text encoder, distilled to run in 8 steps without classifier-free guidance. It renders photoreal scenes and legible text, follows long prompts closely, and understands English and Chinese. The app loads it at start-up and uses it whenever the prompt names no model and no style word points at a specialist. On a 24 GB card the loader enables per-model CPU offload so the text encoder leaves the GPU after encoding and the 12 GB transformer has room for activations. Like FLUX it has no negative prompt and no guidance scale, and LoRAs are not applied to it; prompt weighting is also not available because its text encoder is not CLIP.

6.8 Qwen-Image on 24 GB

Qwen-Image (Alibaba, Apache 2.0) is a 20-billion-parameter MMDiT with a Qwen2.5-VL-7B text encoder and the strongest open-weight text rendering and layout control. Its bf16 weights are 41 GB, so on a 24 GB card the loader quantises the transformer to 4-bit NF4 shard by shard straight onto the GPU on first use and saves the result under models/cache/; the text encoder comes pre-quantised from the diffusers NF4 repository. The

Lightning 8-step LoRA from lightx2v is attached as a permanent adapter with the exponential time-shift scheduler it was distilled for, which replaces the stock 50 steps with classifier-free guidance by 8 steps without it. User LoRAs stack on top of Lightning. The model has no negative prompt in this configuration and the guidance word scaling is ignored.

6.9 FLUX.1 schnell on 24 GB

FLUX.1 is a 12-billion-parameter flow-matching transformer. Its bf16 weights alone are 24 GB, so on a card under 40 GB the loader quantises the transformer to 4-bit NF4 and the T5 text encoder to 8-bit with bitsandbytes, then enables per-model CPU offload. Expect a few minutes of load time and about 15 to 25 s per 1024x1024 image. The repository is gated: the loader raises a clear error with the login instructions if the terms have not been accepted. This path is implemented but has not been exercised on the development machine because no Hugging Face token was available.

7. Prompt Language Reference

Everything about a generation is expressed in the prompt. Matching is case-insensitive. Model and LoRA tags and numeric directives are removed before the prompt reaches the model; descriptive words stay in because they also guide the image.

7.1 Model and LoRA

Write	Effect
model zimage, model juggernaut, model: flux, checkpoint=realvis	Use that model (key, alias, name, repo id or local file name)
using realvis, with dreamshaper, via flux, on playground	Same, for known model words only
<lora:name:0.8>, <lora:name>	Activate models/loras/name.safetensors at that weight (default 1)
(word:1.3), (word)++, (word)--, (word:0.7)	Emphasis weighting, kept in the prompt and applied by compel

7.2 Size and shape

Write	Effect	Generated at
16:9	Widescreen	1344x768
9:16	Vertical widescreen	768x1344
4:3 / 3:4	Classic	1152x896 / 896x1152
3:2 / 2:3	Photo	1216x832 / 832x1216
1:1	Square	1024x1024
21:9 / 9:21	Ultrawide	1536x640 / 640x1536
5:4 / 4:5	Print	1088x896 / 896x1088
1920x1080 (any WxH)	Same ratio, scaled to about 1 megapixel; if larger than native, the output is upscaled and cropped to exactly that size	depends
portrait, tall, vertical, poster, book cover	2:3	832x1216
landscape, wide, horizontal	3:2	1216x832
square, icon, avatar, profile picture	1:1	1024x1024
wallpaper, desktop wallpaper, widescreen, banner, youtube thumbnail	16:9	1344x768
phone wallpaper, mobile wallpaper, instagram story, tiktok	9:16	768x1344
ultrawide, cinemascope	21:9	1536x640

Precedence: an explicit WxH beats an explicit ratio, which beats orientation words. Among orientation words the first match in the table order wins, so "portrait of a king, landscape" is tall.

7.3 Output resolution (upscaling)

Write	Long side after upscaling	Passes
2k	2560 px	1
4k or uhd	3840 px	1
8k	7680 px	2
3840x2160 (any size above native)	exactly that size, centre-cropped	1 or 2

Any of these also raises steps by about a third unless steps are given explicitly. The same targets are offered by the Upscale bar after generation.

7.4 Quality and fidelity

Write	Effect
hd, ultra detailed, highly detailed, masterpiece, best quality, high quality	steps x 1.35 (SDXL: 40)
quick, draft, fast, rough, preview	steps x 0.5 (SDXL: 15)
25 steps, steps 25, steps=25	exactly that (1 to 100)
strict, exact, precise, literal	guidance x 1.3 (SDXL: 9.1); ignored by FLUX
creative, loose, dreamy, abstract, surreal	guidance x 0.7 (SDXL: 4.9); ignored by FLUX
cfg 8, guidance 8	exactly that (0 to 20)

Defaults come from the model: 30 steps and guidance 7 for SDXL base, see section 6.1 for the others.

7.5 Style presets

The first matching style adds descriptors that are not already present, and may auto-select a model (section 6.2).

Write	Descriptors added	Auto model
photorealistic, photo, photograph, dslr	photorealistic, 8k, DSLR photo, sharp focus, natural lighting	Juggernaut
anime, manga	anime style, vibrant colors, clean lines, studio quality	Animagine
cinematic, film still, movie scene	cinematic lighting, dramatic, film still, depth of field, color graded	Juggernaut
watercolor	watercolor painting, soft edges, paper texture, pastel tones	Playground
oil painting, oil on canvas	oil painting, visible brush strokes, canvas texture, classical	Playground
3d render, 3d, octane, blender, unreal	3d render, octane render, volumetric lighting, highly detailed	-
pixel art, 8-bit, 16-bit	pixel art, retro game sprite, crisp pixels	-
digital art, concept art, digital painting	digital painting, concept art, trending on artstation, highly detailed	DreamShaper
sketch, pencil sketch, line art, ink drawing	pencil sketch, line art, monochrome, hand drawn	-
comic, cartoon	comic book style, bold outlines, flat colors, halftone	DreamShaper
cyberpunk	cyberpunk, neon lights, rain, futuristic city, high contrast	DreamShaper
low poly	low poly, geometric, isometric, minimal shading	-

The photorealistic preset contains the word "8k" as a quality descriptor; it does not trigger 8K upscaling because upscale words are detected on the original prompt before the suffix is added.

7.6 Batch and reproducibility

Write	Effect
3 images, 4 variations, 2 versions, x4	1 to 4 images, seeds N, N+1, N+2 ...
seed 42, seed: 42, seed=42	fixed seed; omitted means random

7.7 Worked examples

Prompt	Interpreted as
a cozy cabin in a snowy forest, 16:9, 4k	1344x768, upscale to 3840x2194, 40 steps, guidance 7, loaded model

Prompt	Interpreted as
close-up portrait of an old fisherman, photorealistic	832x1216, 30 steps, guidance 5, Juggernaut XL v9 (auto)
a knight in silver armour, (glowing sword:1.3), model realvis, quick	1024x1024, 15 steps, guidance 4, RealVisXL V5.0, sword emphasised
neon city street at night using flux, 16:9	1344x768, 4 steps, guidance 0, FLUX.1 schnell
cat astronaut <lora:pixel-style:0.7> with dreamshaper	1024x1024, 6 steps, guidance 2, DreamShaper, LoRA pixel-style x0.7
anime girl with umbrella, tall, strict, 2 variations	832x1216, 28 steps, guidance 7.8, Animagine (auto), 2 images
oil painting of a ship in a storm 21:9 steps 45 cfg 8	1536x640, 45 steps, guidance 8, Playground (auto)

8. Database

8.1 Overview

File: generations.db in the project folder. Engine: SQLite 3 through Python's standard library. One table, generations, one row per image produced by the web app. An index on created_at supports newest-first listing. The schema is created on first connection; columns added in later increments are listed in NEW_COLUMNS and added with ALTER TABLE when absent. Since v2.0 rows can be amended after creation: the Upscale bar writes the upscale columns onto an existing row with db.update().

8.2 Table: generations

Identity and files

Column	Type	Meaning
id	INTEGER PK	Auto-increment row id, shown as "#N" in the UI
created_at	TEXT	ISO-8601 local timestamp, seconds precision
image_path	TEXT	Absolute path of the native PNG
file_size_kb	REAL	Size of the native PNG

Prompt

Column	Type	Meaning
user_prompt	TEXT	Exactly what was typed, including model and LoRA tags
final_prompt	TEXT	What was sent to the model after directive removal and style suffixes
negative_prompt	TEXT	Negative prompt used; empty for FLUX
parser_notes	TEXT	Parser decisions, auto-model note, LoRA notes and the sampler, joined with " - "

Settings

Column	Type	Meaning
model_id	TEXT	Model source: Hugging Face repo id or local checkpoint path
width, height	INTEGER	Native generation size
steps	INTEGER	Denoising steps actually used (after model defaults)
guidance	REAL	Guidance scale actually used
seed	INTEGER	Seed used for this image
style	TEXT	Name of the matched style preset, or NULL
batch_index, batch_size	INTEGER	Position within a multi-image request

Timing

Column	Type	Meaning
inference_seconds	REAL	Pipeline call, from start to CUDA synchronise
save_seconds	REAL	PNG encode and write of the native image
total_seconds	REAL	inference + save (excludes upscaling)
steps_per_second	REAL	steps / inference_seconds

GPU and software

Column	Type	Meaning
gpu_name	TEXT	From torch device properties
vram_total_gb	REAL	Card memory
vram_peak_gb	REAL	Peak torch allocation during this image, including an in-prompt upscale
vram_reserved_gb	REAL	Allocator reservation after the image
compute_capability	TEXT	e.g. "8.6"
driver_version	TEXT	From nvidia-smi
cuda_version, torch_version, diffusers_version	TEXT	Library versions

Telemetry (from nvidia-smi)

Column	Type	Meaning
temp_before_c, temp_after_c	REAL	GPU temperature before the pipeline call and right after it
power_before_w, power_after_w	REAL	Board power draw at the same two moments
power_limit_w	REAL	Configured power limit
gpu_util_after_pct	REAL	GPU utilisation at the after snapshot
sm_clock_mhz, mem_clock_mhz	REAL	Clock speeds at the after snapshot
fan_after_pct	REAL	Fan speed
pstate	TEXT	Performance state, P0 to P8

Upscaling (written at generation or later by the Upscale bar)

Column	Type	Meaning
upscale_target	INTEGER	Requested long side in pixels
upscaled_width, upscaled_height	INTEGER	Final size after resample and crop
upscaled_path	TEXT	Absolute path of the upscaled PNG
upscaled_size_kb	REAL	Size of that file
upscale_seconds	REAL	Time of the upscaling stage
upscale_passes	INTEGER	Number of x4 passes (1 or 2)
upscaler	TEXT	Architecture and weight file name

Video and music (v2.2)

Column	Type	Meaning
kind	TEXT	image, video or music; NULL on rows written before v2.2 means image. image_path holds the .mp4 or .wav for the other kinds
duration_s	REAL	Length of the clip or track in seconds
fps	REAL	Frames per second of a clip (24 for LTX-Video, 16 for Wan)
frames	INTEGER	Number of frames in a clip
sample_rate	INTEGER	Sample rate of a track (32000 for MusicGen, 44100 for Stable Audio)
soundtrack_path	TEXT	The _music.mp4 written when a track was laid under the clip
category	TEXT	Category chosen in the Studio or by batch.py (v2.3); the file then lives in outputs/<category>/

db.counts_by_kind() returns count, today, GPU seconds and total duration per kind; fetch_filtered() takes a kind filter; stats_by_model() counts clips and tracks like images (the Admin table calls the column uses).

Catch-all

Column	Type	Meaning
extra_json	TEXT	JSON of any record keys that have no column, for forward compatibility

Table: prompt_library (v2.3)

Every prompt generated under a category, one row per generation, so the prompts of a category can be listed, exported or reused without the images. Written by app.generate() when a category is set (the Studio box or batch.py); read by db.prompts(), db.categories() and the History category filter.

Column	Type	Meaning
id	INTEGER	Primary key
created_at	TEXT	ISO timestamp
category	TEXT	Category name as typed (case-insensitive matching)
prompt	TEXT	The prompt as written by the user (including hints such as 16:9 or seed 1001)
kind	TEXT	image (video and music do not use categories yet)
source	TEXT	studio (the Studio box and batch.py both go through the app)
generation_id	INTEGER	The generations row the prompt produced

8.3 Example queries

Average inference time by model:

```
SELECT model_id, COUNT(*) AS n, ROUND(AVG(inference_seconds), 2) AS avg_s,
       ROUND(AVG(steps_per_second), 2) AS steps_s
FROM generations GROUP BY model_id ORDER BY n DESC;
```

Hottest runs:

```
SELECT id, temp_after_c, power_after_w, user_prompt
FROM generations ORDER BY temp_after_c DESC LIMIT 5;
```

Every 4K or larger image with its file:

```
SELECT id, upscaled_width, upscaled_height, upscale_seconds, upscaled_path
FROM generations WHERE upscale_target >= 3840;
```

Estimated GPU energy of all generations, as the Admin panel computes it:

```
SELECT ROUND(SUM(inference_seconds * (power_before_w + power_after_w) / 2) / 3600, 1) AS wh
FROM generations;
```

8.4 Access from Python

```
import db
rows = db.fetch(limit=10, search="cabin")
one = db.get(4)
db.update(4, {"upscaled_width": 3840})
totals = db.stats()
```

9. Admin Panel

The Admin tab has two jobs: it manages the studio (models, LoRAs, downloads, settings) and it watches the machine. Everything an admin changes is stored in models/registry.json; everything the panel measures is sampled on demand and never stored. The eight sub-tabs:

Sub-tab	Purpose
Overview	System tiles (9.4), activity tiles (images today and this week, GPU minutes in the last 24 h, loaded and start-up model, models on disk, LoRAs, downloads, outputs size, Hugging Face login) and the per-model statistics table (images, average, fastest and slowest seconds, steps per second, average and maximum peak VRAM, hottest temperature, upscaled count, last use). Refreshes on open, on Refresh and every 5 s when ticked.
Models	Table of every model: key, name, family, type (built-in, local file, added), default steps and guidance, sampler, on disk, size, images, average seconds, enabled, start-up star. Selecting a row fills the editor (9.1).
LoRAs	Table of every LoRA file: name, file, family, trigger words, weight, size, enabled, recorded uses, source. Selecting a row fills the editor (9.2).
Add	Add a model or LoRA from a source (9.3).
Downloads	Every background download with state, bytes, speed, remaining time and the last message; Cancel by number; Clear finished. Refreshes every 3 s.
System	The nine live tiles and six detail sections (9.4 and 9.5).
Settings	Start-up model; default weight for LoRAs ticked in the LoRAs box; default negative prompt; VRAM release after each image on the big flow models; Hugging Face token (saved through huggingface_hub, with Check login); Civitai token; clean-up of generations and images older than N days; admin password lock.
Logs	The app's own log: last N lines with a contains filter, an errors-only level and 3 s auto-refresh.

9.1 Model editor and actions

Field or button	Effect
Name, Description	Shown in the Model box and the model cards.
Family, Source	Editable for added models only; built-ins and local files keep theirs.
Default steps, Default guidance, Sampler, Weight variant	The model's defaults; quality and fidelity words in prompts scale relative to them. Sampler choices: DPM++ 2M Karras, DPM++ SDE Karras, Euler a, Euler, model default.
Aliases, Auto-pick for styles	Extra words that name the model in a prompt; style names for which Auto chooses it when an SDXL-family model is loaded.
Supports a negative prompt, Enabled	Whether the negative box applies; whether the model is listed and usable.
Save changes	Writes the entry (overrides for a built-in, the full spec for an added model). Applies to the next load.
Load now	Loads the model immediately under the GPU lock, downloading first if needed.
Download in background	Queues a download job without loading.
Set as start-up model	The model loaded when the app starts.
Unload current model	Frees the VRAM; the next generation loads again.
Remove from list / reset built-in	Deletes an added model's entry (and unloads it if loaded) or resets a built-in to its defaults. Files stay.
Delete files from disk	With the confirmation ticked: removes the local file or the repo's Hugging Face cache folder and the converted caches under models/cache/.

9.2 LoRA editor and actions

Field or button	Effect
Display name	Shown in the LoRAs box; the prompt tag still uses the file name.
Trained for (family)	Auto switches to a model of this family when the LoRA is requested; on a model of another family the LoRA is skipped with a note rather than loaded.
Trigger words	Shown in the LoRAs box and reminded in the "Will generate" line when missing from the prompt. Civitai sources fill them automatically.
Default weight	Used when the LoRA is ticked in the LoRAs box; a prompt tag such as <lora:name:1.2> overrides it.
Description, Source, Tags, Enabled	Notes; where it came from; free tags; whether it is listed.
Try it in the Studio	Fills the prompt with the trigger words, the LoRA tag, a matching model and a fixed seed, and switches to the Studio tab.
Delete LoRA file	With the confirmation ticked: deletes the file and its metadata; if the adapter is inside the loaded pipeline the model is unloaded so the next load is clean.

9.3 Sources the Add tab understands

Source	Example	What happens
Hugging Face repo, diffusers layout	RunDiffusion/Juggernaut-XL-v9	model_index.json names the pipeline class; StableDiffusionXLPipeline, ZImagePipeline, QwenImagePipeline and FluxPipeline map to the four families. The fp16 variant is used for SDXL when present. Added as a model; download of the needed weights only (about 6.9 of 13.1 GB for Juggernaut).
Hugging Face repo of single files	ostris/crayon_style_lora_sdxl	The largest .safetensors file is selected (others selectable); under 1.5 GB it is treated as a LoRA, above as a checkpoint.
Hugging Face file	nerijs/pixel-art-xl/pixel-art-xl.safetensors or its https link	That file; the stored Hugging Face token is sent for gated repos.
Civitai	https://civitai.com/models/133005?modelVersionId=782002, a model-versions link or an api/download link	The Civitai API supplies type (Checkpoint or LoRA), base model (SDXL 1.0, Pony, Illustrious, FLUX.1 ...), trigger words, file name and size and the NSFW flag; the Civitai token from Settings is used when present.
Direct URL	https://example.com/model.safetensors	HEAD request for name and size; type guessed from the size.
Local path	D:\downloads\model.safetensors	Copied into models/.
Upload	a file chosen in the browser	Moved into models/ (suited to LoRAs; the limit is 8 GB).

Single files are always loaded with the SDXL single-file loader, so FLUX, Qwen-Image and Z-Image additions must be diffusers repos. File downloads resume after an interruption (the .part file is kept) and refuse answers that are web pages, which is what a missing token looks like. Registration happens when the file has arrived, so an interrupted download leaves nothing half-registered.

9.4 System tiles

Tile	Value	Meter or state	Source
GPU load	utilisation %, SM clock, P-state	meter	nvidia-smi
VRAM used	GiB used of total; torch allocation	meter of used %	nvidia-smi, torch
GPU temperature	°C and fan %	state: OK under 70, Warm under 83, Hot under 90, Critical above	nvidia-smi
GPU power	W of limit, % of limit	meter	nvidia-smi

Tile	Value	Meter or state	Source
CPU	usage %, threads, frequency	per-core bar strip	psutil
Memory	GB used of total	meter	psutil
System disk	GB free, % used	meter	psutil
Network	download and upload rate, established connections	-	psutil, two samples
App uptime	uptime, PID, resident memory	-	psutil

Meters use the pink accent and switch to the amber warning colour above 85 % and the red critical colour above 95 %. The temperature state always carries an icon and a word, never a colour alone.

9.5 System sections

Section	Fields
GPU	Card, VRAM, compute capability and SM count; driver, CUDA, cuDNN; current and maximum SM and memory clocks; GPU and memory-bus utilisation; VRAM from nvidia-smi and from torch (allocated, reserved, session peak); PCIe generation and width; throttle reasons decoded from the nvidia-smi bitmask (idle, power cap, thermal slowdown and so on); VBIOS and PCI bus; loaded model and sampler; processes on the GPU.
CPU	Processor name, architecture, physical and logical cores, current and maximum frequency, total and per-core usage, 1/5/15-minute load averages, process and thread counts, boot time and uptime.
Memory and storage	RAM used, available and total; swap or page file; disk reads and writes since boot; outputs folder size and PNG count; models folder; Hugging Face cache size and path; database size; every partition with file system, total, used, free and percentage.
Network	Hostname and primary IP; current throughput (rates need two samples); bytes and packets since boot; errors and drops; established connections and listening ports; every interface with state, IPv4, MAC, link speed, sent and received; optional reachability of huggingface.co, github.com and pypi.org in milliseconds.
Power	GPU draw against the configured limit with percentage; default and maximum limits; performance state and fan; the active Windows power plan; battery if present; estimated GPU energy of all recorded generations in Wh.
Software and app	OS, Python, torch, CUDA, cuDNN, diffusers, transformers, gradio versions; whether the process runs in Docker; app PID, start time, uptime, resident memory and threads; working directory; generation totals from the database.

9.6 Reading the numbers

- Idle readings on the RTX 3090 are about 45 W, P8, SM clock 210 MHz, PCIe gen 1 x1 and throttle reason "GPU idle". Under load expect P0, 1.7 to 1.9 GHz, PCIe gen 3 or 4 x16 and 350 to 390 W.
- nvidia-smi reports allocator reservation as used VRAM; the torch allocated figure is the live tensor total.
- Process names on the GPU may show "Insufficient Permissions" for processes of other users or the system.
- The network rate tiles show 0 on the first sample after start; every later refresh shows the rate since the previous one.
- Inside Docker the CPU, memory and network sections describe the container, while GPU and power come from the host driver.
- A System or Overview refresh costs about 0.7 s (3 s the first time, when folder sizes are computed) and runs on its own queue, so it keeps working while an image is generating.

9.7 Locking the panel

Deleting model or LoRA files and cleaning outputs always need their confirmation box. When the environment variable FRAZILE_ADMIN_PASSWORD is set, those actions also need the panel unlocked once per browser session with that password in Settings. Set it whenever the server is reachable from other machines (Docker, or

GRADIO_SERVER_NAME=0.0.0.0); on localhost it is optional.

10. Video and Music

10.1 Overview

Version 2.2 adds two model families next to the image families. They are ordinary registry entries, so the Admin panel lists, edits, downloads, loads and unloads them like any image model, and the Add tab accepts any LTX-Video, Wan, MusicGen or Stable Audio repository. They are used from their own tabs because their controls (length, frame rate, start image, temperature) have no meaning for images.

Model	Key	Family	Loader	Output	Limits	Download
LTX-Video 2B (Lightricks)	ltx	video	ltx	24 fps H.264 MP4	2 to 10 s; 768x512, 512x768, 832x480, 480x832, 1024x576, 576x1024; text- and image-to-video	28 GB
Wan 2.1 T2V 1.3B (Wan-AI)	wan	video	wan	16 fps H.264 MP4	2 to 6 s; same sizes, 832x480 native; text-to-video only	29 GB
MusicGen stereo medium (Meta)	musicgen	music	musicgen	32 kHz stereo WAV	3 to 30 s, instrumental	4.1 GB
MusicGen stereo small	musicgen-small	music	musicgen	32 kHz stereo WAV	3 to 30 s, instrumental	1.3 GB
Stable Audio Open 1.0 (Stability AI)	stable-audio	music	stable_audio	44.1 kHz stereo WAV	3 to 47 s; gated repo, needs a Hugging Face token; untested on the development machine	5.3 GB

The loader column is the architecture media.py detects from the repository (model_index.json for diffusers pipelines, config.json model_type for MusicGen). An admin-added repo gets the same treatment, so a fine-tuned LTX or a MusicGen melody checkpoint works without code changes.

10.2 The Video tab

- Prompt and chips: the model responds to a short description of scene, motion and camera; the chips add tracking shot, drone shot, close-up, time-lapse, slow motion, golden hour, cinematic lighting, rain and fog.
- Model box: LTX-Video 2B or Wan 2.1 (downloaded models first). Changing it resets steps, guidance, the negative prompt, the maximum length and shows or hides the start-image box.
- Size: six presets that both models accept (multiples of 32 for LTX, 16 for Wan). Length: 2 to 10 s; the frame count is rounded to 8k+1 (LTX) or 4k+1 (Wan) frames at the model's frame rate, so 4 s is 97 frames with LTX and 65 with Wan.
- Steps and guidance: LTX 40 and 3.0, Wan 30 and 5.0 by default (editable per model in Admin). Seed: -1 picks a random one; the clip's seed is in its details and History row.
- Start image (LTX only): any picture, including a Studio result dragged in. It is centre-cropped and resized to the chosen size and passed to the image-to-video pipeline, which shares the loaded components.
- Result: the clip plays in the result box; the details report lists timing, size, frames, fps, length, seed, file and the GPU telemetry.
- Upscale video: Full HD (1920), 2K (2560) or 4K (3840) long side. Frames are read back from the MP4, run one by one through Real-ESRGAN's compact video network (realesr-general-x4v3, 1.2M parameters, fetched into models/ on first use; the x4plus image network is the fallback) with a single x4 pass and a Lanczos resample to the exact target, and streamed into a new MP4 at crf 18. A sound track already added is kept.
- Add soundtrack: the box lists the tracks made in the Music tab; the chosen one is muxed under the current clip, cut to its length, with a one-second fade-out, into a new _music.mp4.
- Clear resets the tab; the clip stays in outputs/ and in History.

10.3 The Music tab

- Prompt and chips: genre, mood, instruments and tempo; chips for lo-fi hip hop, cinematic orchestral, synthwave, acoustic folk, jazz trio, techno, ambient pads, epic trailer, upbeat, melancholic, relaxing and 120 bpm.
- Length 3 to 30 s with MusicGen (47 s with Stable Audio); guidance (3 by default) pulls the result towards the text; temperature (1.0) adds variety; seed as in the Video tab.
- MusicGen generates 50 tokens per second of audio autoregressively, so progress is reported per token; a 15 s track is 750 decoder steps.
- The negative prompt box appears only for Stable Audio; MusicGen has no such input.
- Result: a player with the track and the details report; the file is a 16-bit WAV in outputs/.

10.4 Sharing one GPU

The image ModelManager and the MediaManager share one lock, so one job runs at a time, and media.prepare_for() decides what must leave the card before a model is used:

About to use	Image model loaded	Video model loaded	Music model loaded
An image model	kept (swapped as usual)	unloaded	kept, unless the image model is Z-Image, Qwen-Image or FLUX
A video model	unloaded	swapped if different	unloaded
A music model	kept while free VRAM >= need + 2 GB, else unloaded	unloaded	swapped if different

The next image generation reloads the image model automatically, so the user never unloads anything by hand. The header pill and the Loaded now line show both managers, and Admin > Models > Unload frees both.

10.5 How the models are loaded

- The repositories ship their text encoders in fp32: T5-XXL (LTX) is 19 GB and UMT5-XXL (Wan) 23 GB on disk, more than this machine's free RAM. Every part is therefore loaded with device_map onto the GPU shard by shard in bf16, never assembled on the CPU. The first load saves a bf16 copy of the text encoder under models/cache/, which halves later load times.
- LTX: T5EncoderModel + LTXVideoTransformer3DModel + AutoencoderKLLTXVideo with tiling, about 14 GB resident. Wan: UMT5EncoderModel in 8-bit on cards under 32 GB (its 11 GB bf16 form plus the fp32 VAE decode paged VRAM on the 24 GB card and took over ten minutes) + WanTransformer3DModel + fp32 AutoencoderKLWan with tiling and the UniPC scheduler (flow_shift 3.0).
- MusicGen: MusicgenForConditionalGeneration in fp16 with its processor, 4.1 GB resident; Stable Audio: StableAudioPipeline in fp16.
- Downloads fetch only the diffusers sub-folders (model_index.json and */*). This matters for LTX, whose repository also holds about 250 GB of single-file checkpoints for other runtimes.

10.6 Files, thumbnails and the database

- Clips: outputs/<timestamp>_seed<seed>.mp4; upscales add _up<px>; soundtracks add _music. Tracks: outputs/<timestamp>_seed<seed>.wav.
- Thumbnails in outputs/.thumbs/: the first frame (JPEG) for clips, a waveform picture (PNG) for tracks.
- generations.db rows carry kind = video or music plus duration_s, fps, frames, sample_rate and soundtrack_path; the Video tab's upscale fills the usual upscale columns. The History tab filters by kind and shows the right player; Reuse prompt returns to the Video or Music tab.

10.7 Measured on the RTX 3090

Operation	Settings	Time	VRAM
LTX-Video 2B, first load	fp32 to bf16 conversion of T5-XXL, bf16 copy written to models/cache/	186 s	14.2 GB resident
LTX-Video 2B, later loads	from the bf16 copy (inside the app)	94 s	14.2 GB resident
LTX text-to-video	768x512, 89 frames (3.7 s), 30 steps, through the app	29.9 s + 1 s encode	15.5 GB peak
LTX text-to-video	768x512, 49 frames (2 s), 20 steps	11.4 s	17.0 GB reserved
LTX image-to-video	768x512, 65 frames (2.7 s), 20 steps, from a 4K Studio PNG, through the app	15.7 s	14.9 GB peak
LTX image-to-video	768x512, 49 frames, 20 steps	12.9 s	15.3 GB peak
Wan 2.1 1.3B, first load	UMT5-XXL converted to bf16 and cached, then quantised to 8-bit	266 s	11.1 GB resident
Wan 2.1 1.3B, later loads	bf16 copy quantised to 8-bit	164 s	11.1 GB resident
Wan text-to-video	832x480, 49 frames (3 s), 30 steps, UniPC	205 s (about 6 s per step)	12.9 GB peak
Video upscale to 4K	65 frames 768x512 to 3840x2560, compact Real-ESRGAN, one pass + resample, H.264 crf 18	49 s (0.76 s per frame)	under 1 GB extra
Video upscale to 4K (fallback)	same clip with the x4plus image network	135 s one pass, 204 s two passes	under 1 GB extra
Soundtrack mux	10 s WAV under a 2.7 s clip, AAC, fade-out	0.4 s	none
MusicGen stereo medium, load	fp16, from the Hugging Face cache	24 to 27 s	4.1 GB resident
MusicGen stereo medium, track	10 s, guidance 3, temperature 1.0, through the app	36 s	4.2 GB peak
MusicGen stereo medium, track	6 s, guidance 3	23.6 s	4.6 GB reserved
Image after a video	Juggernaut XL v9 reloaded automatically after LTX-Video was unloaded	31 s load + 5.0 s image	10.4 GB peak

Measured on 22 October 2026 with torch 2.14.1, diffusers 0.41 and transformers 5.20. Video generation time grows roughly linearly with frames x steps; 768x512 at 97 frames and 40 steps is the recommended quality setting and takes about four times the 49-frame, 20-step figure.

10.8 Limits and tips

- Clips are short by design (2 to 10 s). For longer sequences, generate the next clip from the last frame of the previous one (save a frame, use it as the start image) and join the files with ffmpeg.
- LTX responds to specific, literal descriptions of motion; abstract prompts give static camera moves. Keep people small in frame or use close-ups; hands and faces in mid-shots are the weakest area of 2B-class models.
- Wan's native size is 832x480; other sizes work but cost more time. It has no image-to-video in this 1.3B variant.
- 4K from a 768x512 clip is a 5x enlargement by a super-resolution network, not native detail. It is sharp and clean (Real-ESRGAN's video network, chosen over the x4plus image network because it is about 5x faster per frame and steadier between frames) but cannot invent texture that the 2B model did not render; generate at 1024x576 first when the subject allows.
- MusicGen is instrumental only and best under 30 s; describe instruments and tempo rather than song titles or artists.

11. Performance

All figures were measured on the development machine (RTX 3090, 24 GB, driver 610.88, torch 2.14.1 with CUDA 12.6) and taken from generations.db and the v2.0 model sweep.

11.1 Generation by model (832x1216, prompt weighting on for SDXL)

Model	Steps	CFG	Sampler	Inference	Peak VRAM
Z-Image Turbo	8	0	Flow-match Euler	8.6 s (9.8 s at 1344x768)	18.1 GB
Qwen-Image + Lightning (4-bit)	8	1 (none)	Flow-match Euler, exp. shift	13.4 s (13.3 to 14.2 s at 1344x768 and 1024x1024)	18.0 GB
SDXL base 1.0	30	7	DPM++ 2M Karras	8.0 s	13.1 GB
Juggernaut XL v9	30	5	DPM++ 2M Karras	8.0 s	13.1 GB
RealVisXL V5.0	30	4	DPM++ SDE Karras	8.0 s	13.2 GB
DreamShaper XL v2 Turbo	6	2	DPM++ SDE Karras	2.0 s	13.2 GB
Playground v2.5	40	3	EDM DPM-Solver	11.5 s	13.2 GB
Animagine XL 3.1	28	6	Euler a	7.3 s	13.2 GB

Every SDXL-family model runs at about 3.7 to 3.8 steps per second at this size. Inference time is therefore proportional to the step count, which is why the turbo model's 6 steps finish in 2 seconds. Z-Image Turbo runs at about 1.1 s per step but needs only 8, so a finished image takes 9 to 10 s with better prompt adherence and legible text. Loading an SDXL-family model from the local cache takes 26 to 27 s; freeing it returns resident VRAM to 0.01 GB. Measurements taken while another process holds GPU memory are not valid: with the web app resident alongside the test, Z-Image took 78 to 128 s per image because Windows paged VRAM.

11.2 Generation (v1.0 measurements, 1024x1024 and 16:9, SDXL base)

Size	Steps	Inference	Steps/s	Peak VRAM	Temp after	Power after
1024x1024	15	4.7 to 4.9 s	3.1 to 3.2	11.5 GB	55 to 56 C	377 to 379 W
1024x1024	30	9.7 s	3.1	11.5 GB	-	-
1344x768	40	12.8 to 13.5 s	3.0 to 3.1	11.4 GB	52 to 54 C	349 to 357 W
768x1344	40	11.2 to 11.4 s	3.5 to 3.6	11.5 GB	55 to 57 C	382 to 385 W

11.3 Upscaling

From	To	Passes	Time	Output file
1344x768	3840x2194 (4K)	1	3.4 s	9.9 MB
768x1344	2194x3840 (4K)	1	2.9 s	7.9 MB
768x1344	1463x2560 (2K, Upscale bar)	1	3.6 s	3.5 MB
768x1344	4388x7680 (8K)	2	8.4 to 8.5 s	30.5 MB

11.4 Memory

Stage	VRAM
Idle with one SDXL-family model loaded	6.6 GB allocated
Idle with Z-Image Turbo loaded (bf16 transformer + 8-bit text encoder)	15.8 GB allocated

Stage	VRAM
Idle with Qwen-Image loaded (NF4 transformer + 4-bit text encoder + Lightning LoRA)	17.5 GB allocated
Peak during Qwen-Image generation at 1 MP (VAE tiling on)	18.0 GB; 21.7 GB before VAE tiling, which caused paging
Peak during Z-Image generation at 1344x768	18.1 GB
Peak during 1 MP generation, plain prompt	11.4 to 10.5 GB
Peak during 832x1216 generation with compel embeddings	13.1 to 13.2 GB
After unload	0.01 GB
Upscaler weights	67 MB (half precision)
Peak during tiled upscale	no measurable increase over the generation peak

11.5 Startup and switching

Step	Time
First ever start (downloads 6.7 GB)	bandwidth dependent; the five fine-tunes downloaded in 76 to 184 s each on the development connection
SDXL load from local cache at start	34 to 59 s (disk cache dependent)
Z-Image Turbo first load (fp32 to bf16 conversion plus writing the 12 GB cache)	about 250 s
Z-Image Turbo load from models/cache/	about 130 s (8-bit quantisation of the text encoder dominates)
Qwen-Image first load (quantising 41 GB to NF4 and writing the 11 GB cache)	about 260 s; needs most of the 16 GB system RAM
Qwen-Image load from models/cache/	about 90 s
Model switch while running (SDXL family)	26 to 27 s
Upscaler load	under 1 s
Admin panel refresh	about 3 s first, 0.7 s after

11.6 Tuning levers

- Model: Z-Image Turbo gives a finished photoreal image in 9 s; DreamShaper Turbo gives a rougher draft in 2 s. For SDXL-specific looks or LoRAs, regenerate the chosen seed with Juggernaut or RealVis.
- Steps: the dominant cost for every model. `quick` halves them; `highly detailed` adds a third.
- Count: images in a batch are generated sequentially, so time scales linearly.
- Upscale later: the Upscale bar costs 3 to 9 s per image and avoids regenerating.
- Cards with less VRAM: generate.py enables CPU offload below 12 GB, which roughly doubles inference time but keeps the pipeline working.

12. Deployment and Docker

12.1 Image

The Dockerfile builds on `nvidia/cuda:12.6.3-cudnn-runtime-ubuntu24.04`, installs Python 3.12, PyTorch and torchvision for CUDA 12.6, then requirements.txt, and copies the Python sources. The Hugging Face cache is pointed at /cache/huggingface, Gradio is told to bind 0.0.0.0:7860, and FRAZILE_IN_DOCKER=1 stops the app from trying to open a browser. A health check polls the root page every 30 s after a 3-minute start grace.

12.2 Compose service

Setting	Value	Why
ports	7860:7860	The studio on http://localhost:7860
volumes	./outputs, ./models, ./generations.db, hf-cache	Images, your checkpoints and LoRAs, the history database, and the downloaded models all live outside the container
environment	HF_TOKEN	Optional; lets the gated FLUX repository download
deploy.resources.reservations.devices	driver nvidia, count all, capabilities gpu	GPU passthrough
shm_size	2 GB	Shared memory for data loaders
restart	unless-stopped	Comes back after a reboot of the host

12.3 Host requirements

Host	Needs
Windows 11	WSL 2 (`wsl --install`, reboot), Docker Desktop with the WSL 2 backend, the normal NVIDIA Windows driver. GPU support is built into Docker Desktop; no extra toolkit.
Linux	Docker Engine, the NVIDIA driver, nvidia-container-toolkit configured as a Docker runtime.

The development machine has neither WSL nor Docker installed, so the image has been written and reviewed but not built or run there. The first build on any machine downloads several GB of Python packages and should be run on a fast connection.

12.4 Commands

```
docker compose build           # build the image
docker compose up              # run in the foreground
docker compose up -d           # run detached
docker compose logs -f         # follow the app log
docker compose down            # stop; keeps the hf-cache volume
docker compose down -v         # stop and delete cached models
```

12.5 Differences inside the container

- Paths in the database are container paths (/app/outputs/...). Mount the same folders when moving between native and container use, or the History tab shows "not found" for the other side's files.
- The Admin panel's CPU, memory and network sections describe the container's view; GPU and power still come from the host driver through nvidia-smi.
- On Linux hosts create an empty generations.db before the first `up`, otherwise Docker creates a directory in its place.
- There is still no authentication. Publishing port 7860 beyond localhost exposes the studio to the network.

13. Operations and Troubleshooting

13.1 Starting and stopping

Action	How
Start	Double-click run-app.bat, or `.\.venv\Scripts\python.exe app.py` in the project folder; or `docker compose up`
Confirm ready	Console shows "SDXL base 1.0 ready in N s", "Upscaler ready", "Running on local URL: http://127.0.0.1:7860"
Stop	Ctrl+C in the console, or close the window; `docker compose down` for the container
Change port or address	Set GRADIO_SERVER_PORT or GRADIO_SERVER_NAME before starting; FRAZILE_NO_BROWSER=1 suppresses the browser tab
Expose on the LAN	GRADIO_SERVER_NAME=0.0.0.0. There is no authentication, so only do this on a trusted network

13.2 Logs

The app prints one line per loaded model, per generated image and per upscale, for example:

```
Loading Juggernaut XL v9 ...
Juggernaut XL v9 ready in 26.3s  (scheduler: DPM++ 2M Karras)
#12 juggernaut saved 20261022-101040_seed1746058488.png  832x1216  8.0s  peak 13.1 GB  56.0 C
#12 upscaled -> 1463x2560 in 3.56s
```

To keep a log file, redirect: `.\.venv\Scripts\python.exe -u app.py > app.log 2>&1`. Inside Docker use `docker compose logs -f`.

13.3 Known issues encountered during development and their fixes

Symptom	Cause	Fix applied
AttributeError: 'StableDiffusionXLPipeline' object has no attribute 'enable_vae_slicing'	diffusers 0.41 removed the pipeline-level helper	Call pipe.vae.enable_slicing() instead
Parser ignored "seed 42" and misread "21:9 steps 45" as 9 steps	Directive removal used offsets from a stale lowercase copy of the prompt	Rewrote removal to search and cut the live prompt string each time
Regex word boundaries silently vanished	A patch script wrote "\b" through a normal Python string, turning it into a backspace	Patches now use raw strings or the file is rewritten whole
compel: 'EmbeddingsProviderMulti' object has no attribute 'empty_z'	compel's pad_conditioning_tensors_to_same_length does not support SDXL's two encoders, so a long prompt with a short negative failed	models.py pads the shorter embedding with chunks of the empty-prompt embedding
Theme build failed: Base.set() got an unexpected keyword argument 'color_accent_dark'	Not every Gradio theme variable has a dark-mode twin	build_theme() filters keys against the signature of Base.set()
Admin refresh took 3 s every time	Walking the 39 GB Hugging Face cache and every process for thread counts	Folder sizes cached 60 s, thread count cached 30 s; refresh is 0.7 s
Segmentation fault while generating with Z-Image Turbo	enable_model_cpu_offload() with the 23 GB fp32 transformer on a 16 GB RAM machine	Transformer loaded shard by shard onto the GPU in bf16 with device_map; text encoder in 8-bit; bf16 copy cached under models/cache/
Z-Image generation took 78 to 128 s	The web app was still holding 7 GB of VRAM during the benchmark, so Windows paged GPU memory	Stop other GPU processes before benchmarking; with the GPU free the same images take 8.6 to 9.8 s

Symptom	Cause	Fix applied
Second Qwen-Image image took 207 s (first took 14 s)	VAE decode pushed the peak to 21.7 GB of 24 and allocator fragmentation kept it there, so Windows paged VRAM	VAE slicing and tiling (peak 18.0 GB), torch.cuda.empty_cache() after every image for the big flow models, and PYTORCH_CUDA_ALLOC_CONF=expandable_segments; every image now takes 13 to 14 s
"strict" reported higher guidance for FLUX	Fidelity words were noted before the model was known	apply_defaults() notes that the model has no guidance scale
Space before a comma after a removed tag ("cat , quick")	Tag removal left the separator behind	Tidy step also collapses whitespace before commas
pip could not write pymupdf.exe in C:\Python312\Scripts	Permission problem in the system Python's Scripts folder	Not a project issue; the venv is unaffected. The documentation tooling used the system Python only for one-off edits
UnicodeEncodeError when printing reports from scripts	Windows console code page cp1252 cannot print the arrow and dot characters	Set PYTHONIOENCODING=utf-8 when printing reports; the browser is unaffected
torchvision installed as the CPU build	pip pulled torchvision from PyPI before the CUDA index was consulted	Reinstalled from the cu126 index; requirements.txt documents the order

13.4 Troubleshooting guide

Problem	Check	Resolution
Video generation says Out of VRAM	Choose a smaller size or a shorter clip; 1024x576 at 10 s is the practical limit of a 24 GB card. Unload the image model in Admin > Models if it is still resident.	
The Video tab is slow to start the first time	The first load converts the 19 GB fp32 text encoder to bf16 and writes a copy under models/cache/; later loads take about a minute.	
No sound in a clip with a soundtrack	The soundtrack is a separate _music.mp4 file; the player shows it after Add soundtrack. Upscaling after that keeps the audio; upscaling before it does not add any.	
MusicGen output is silent or noisy	Raise the length (very short tracks are mostly intro), lower the temperature towards 0.9, keep guidance at 3; a different seed changes the arrangement entirely.	
"CUDA GPU not available" at startup	`python -c "import torch; print(torch.version.cuda)"` prints None	Reinstall torch from the cu126 index inside the virtual environment
Port 7860 already in use	Task Manager shows a python.exe from a previous run	End that process, or set GRADIO_SERVER_PORT
"Unknown model 'x'" in the preview line	Typo, or a checkpoint added without restarting	Use a key from the Models accordion; restart after adding files to models/checkpoints/
Generation waits 30 s before starting	The progress bar says "Loading <model>"	Normal for a model switch. Keep the Model box on one model, or Auto, for consecutive prompts
First use of a model waits minutes	Progress says "downloading ~6.5 GB"	Wait; check the HF cache folder growth
"FLUX.1 [schnell] is gated"	No Hugging Face token stored	Accept the terms on the model page and run huggingface-cli login (section 4.10)

Problem	Check	Resolution
LoRA has no effect and the notes say "not found"	File name does not match the tag	Compare with the LoRA list in the Models accordion; matching ignores case and punctuation
Out of memory during generation	Other GPU programs running; or a card under 12 GB; or Z-Image or FLUX on under 24 GB	Close other GPU apps; use generate.py which offloads to CPU on small cards; Z-Image and FLUX need 24 GB
Z-Image is very slow (over a minute per image)	Another process holds several GB of VRAM; nvidia-smi shows memory near 24 GB	Close it; Z-Image needs about 18 GB free
Telemetry fields show None	nvidia-smi not on PATH	Reinstall the NVIDIA driver, or add C:\Windows\System32 to PATH
Upscaler weights missing	FileNotFoundError naming models/RealESRGAN_x4plus.pth	Download from the Real-ESRGAN v0.1.0 release into models/
Upscale bar does nothing	No generation in this browser session	Generate first; the bar works on the records of the last generation
History tab empty	New database, or generations.db moved	Generate an image; or restore the file next to db.py
Admin panel shows 0 B/s network	First sample	Refresh again; rates need two samples
Browser slow to show an 8K result	30 MB PNG transfer	Expected; open the file from outputs/ directly
docker compose fails with "could not select device driver"	GPU support not enabled in Docker	Install WSL 2 and Docker Desktop (Windows) or nvidia-container-toolkit (Linux)

13.5 Maintenance

- Outputs grow by 1.5 MB per native image and 3 to 31 MB per upscaled image. Prune outputs/ periodically; the History tab will show "not found" for deleted files. The Admin panel shows the folder size.
- Each built-in model is 6.5 GB in the Hugging Face cache. Delete a models--* folder there to reclaim space; it downloads again on next use.
- Back up outputs/, generations.db and models/ together so paths stay valid.
- To reset history, stop the app and delete generations.db; it is recreated empty on next start.
- To update dependencies, upgrade inside the virtual environment and rerun the parser self-test, models.py, sysinfo.py and one generation per family. Pin versions in requirements.txt if a breaking change is found.

13.6 Security notes

- The server listens on localhost only by default and has no authentication. The Docker image binds all interfaces inside the container; the compose file publishes the port on the host, so treat that like a LAN exposure.
- No telemetry or prompts are sent anywhere. Network access is limited to model downloads from Hugging Face and the optional reachability check in the Admin panel, which only opens a TCP connection.
- The Admin panel reveals hostnames, IP addresses, MAC addresses, listening ports and process names, and can delete files, store tokens and download models. Anyone who can reach the page can use it; set FRAZILE_ADMIN_PASSWORD when the page is reachable beyond localhost.
- Tokens: the Hugging Face token is stored by huggingface_hub in the user profile; the Civitai token is stored in models/registry.json, which is excluded from git.
- Models and LoRAs downloaded from Civitai or arbitrary URLs are third-party files; only .safetensors files are accepted because the format cannot execute code when loaded.
- Prompts are stored verbatim in generations.db. Treat the file with the same care as the images.

14. Change Log and Roadmap

14.1 Change log

Increment 1: command-line generator (21 Oct 2026)

- Created the project folder, virtual environment, and requirements.txt; installed torch 2.14.1+cu126, diffusers 0.41, transformers, accelerate.
- Wrote generate.py with prompt, negative, steps, guidance, size, seed, count and model arguments. First image: 1024x1024 in 9.7 s, peak VRAM 10.5 GB.

Increment 2: web interface

- Added app.py using Gradio 6.30 with a gallery and an info box; model loaded once at startup and kept resident. Added run-app.bat and README.

Increment 3: prompt-driven controls

- Removed style, size, steps and guidance controls. Added prompt_parser.py, the live "Will generate" line and a collapsible help table.

Increment 4: telemetry and history

- Added gpu_info.py and db.py with a 45-column SQLite table; per-image timing and GPU sampling; the History tab.

Increment 5: real 4K and 8K output

- Added upscaler.py (Real-ESRGAN x4plus, tiled, multi-pass); parser upscale words; eight upscale columns; 4K in 3.4 s, 8K in 8.5 s.

Documentation 1.0 and repository (22 Oct 2026, 09:17)

- Documentation set published as PDFs under Frazile AI ownership. Project pushed to github.com/Fzhacking/ai-image-gen (private).

Increment 6: model registry (22 Oct 2026, 10:09)

- Added models.py: SDXL base, Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo, Playground v2.5, Animagine XL 3.1 and FLUX.1 schnell; local checkpoints; LoRAs; compel weighting; per-model samplers and defaults; ModelManager with verified VRAM release.
- Parser: model and LoRA directives, explicit-flag tracking, apply_defaults(). App: Model dropdown with Auto, model in details and history. CLI: --model, --lora, --list-models.
- Installed peft 0.21, compel 2.5, bitsandbytes 0.50, sentencepiece, protobuf. Downloaded the five fine-tunes (32.5 GB).
- Fixed: compel padding for SDXL dual encoders; comma tidy-up; FLUX fidelity note.
- Measured: every SDXL-family model at 832x1216 in 2.0 to 10.5 s, peak 13.2 GB, swap 26 s.

Increment 7: Frazile AI branding and Upscale bar (22 Oct 2026, 10:27)

- Dark studio theme with pink-to-yellow accents, applied to both colour modes; branded header; Inter and JetBrains Mono.
- Upscale bar after generation: 2K / 4K / 8K, selected image or whole batch, records updated in place with the new db.update().

Increment 8: full width, Admin panel, Docker (22 Oct 2026, 10:50)

- Full-width layout, version pills, Clear button, "Loaded now" line.

- Admin tab with nine tiles and six sections; sysinfo.py collector on psutil and nvidia-smi; 5 s auto-refresh; optional reachability check; caching so a refresh costs 0.7 s.
- Dockerfile, docker-compose.yml and .dockerignore; app reads bind address, port and browser flag from the environment.

Increment 9: Z-Image Turbo as the default model (22 Oct 2026)

- Added the zimage family to models.py with a bf16 loader and CPU offload under 32 GB; Z-Image Turbo (Tongyi-MAI, 6B, 8 steps, Apache 2.0) is the model loaded at start-up.
- The repository ships the transformer in fp32 (23 GB); the first load converts it to bf16 and saves a 12 GB copy under models/cache/ so later loads take about about 130 s. The Qwen3 text encoder is loaded in 8-bit; everything stays on the GPU (15.8 GB resident, 18.1 GB peak).
- Measured: 1344x768 in 9.8 s and 832x1216 in 8.6 s at 8 steps; the FRAZILE AI neon-sign test rendered the text correctly.
- Auto no longer switches away from Z-Image or FLUX; style-based specialist selection applies only while an SDXL-family model is loaded.
- Fixed: CPU offload of Z-Image crashed the process on the 16 GB machine (fp32 shards plus offload exceeded system RAM); replaced by shard-by-shard loading straight onto the GPU.

Increment 10: starter LoRAs and Qwen-Image (22 Oct 2026)

- Pixel Art XL and Papercut LoRAs fetched by models.py fetch-loras; LoRAs box in the UI; with Auto, a requested LoRA switches to an SDXL model so it takes effect.
- Qwen-Image added: 20B transformer quantised to NF4 on first load and cached (11 GB), 4-bit text encoder from diffusers/qwen-image-nf4, Lightning 8-step LoRA as a permanent adapter with the exponential-shift scheduler.
- Measured: 13 to 14 s per 1 MP image at 8 steps, 18.0 GB peak, 90 s cached load. Posters, neon signs and menus render their text correctly.
- Fixed: VRAM paging on the second image (VAE tiling, cache release, expandable segments); FLUX gated-repo error now reported clearly.

Documentation 2.0 (22 Oct 2026)

- Document set rebuilt from docs-src/build_pdf.py with three new sections (Models, Admin panel, Deployment) and a model comparison figure.

Increment 11: model and LoRA management, Studio and History upgrades (22 Oct 2026)

- registry.py: models/registry.json with settings, overrides of built-in models, admin-added models and LoRA metadata (family, trigger words, default weight, source, enabled). models.py merges it into all_models(), family_models(), find_lora() and lora_info(); apply_loras() refuses LoRAs of another family.
- downloads.py: inspect_source() for Hugging Face repos and files, Civitai links, direct URLs and local paths; JobManager with a two-worker pool, byte progress, speed, ETA, resume and cancel; snapshot downloads with per-family ignore patterns; delete_model_files().
- admin_ui.py: the Admin tab rebuilt with Overview, Models, LoRAs, Add, Downloads, System, Settings and Logs; model and LoRA editors; load, unload, download, start-up, remove and delete actions; Hugging Face and Civitai tokens; clean-up; admin password lock. applog.py keeps the app's output for the Logs tab.
- Studio: quick-hint chips, live header pills (loaded model, VRAM, temperature, load, downloads), LoRA trigger-word reminders and family notes in the interpretation line, model cards, empty state, Reuse this seed and New variation; dropdowns refresh when the tab is opened so Admin additions appear without a restart.

- History: model and upscaled-only filters, a thumbnail grid backed by a 384 px JPEG cache (outputs/.thumbs/), Reuse prompt with or without seed, delete with optional file removal. db.py gained delete(), stats_by_model(), recent_counts(), lora_usage(), models_used() and fetch_filtered().
- Concurrency: ModelManager.lock plus the "gpu" and "stats" queue ids, so telemetry refreshes while an image is generating.
- Verified through the Gradio API: chips, preview, overview and system refresh, source inspection, LoRA edit, a LoRA generation that routed Auto from Z-Image to Juggernaut (6.8 s at 1024x1024, 15 steps), history with 18 thumbnails, a 46 MB LoRA (ostris/crayon_style_lora_sdxl) added through the Add flow with a background download, trigger words recorded and used in a generation, and a diffusers test repo registered, fetched and deleted again from the Models tab.

Documentation 2.1 (22 Oct 2026)

- This document set: Admin panel section rewritten around management, module reference for the six new modules, requirements FR-21 to FR-25 and NFR-11 to NFR-12, workflows for adding and managing models.

Increment 12: video and music (22 Oct 2026)

- Two new model families, video and music, with five built-in models: LTX-Video 2B, Wan 2.1 T2V 1.3B, MusicGen stereo medium and small, Stable Audio Open 1.0 (gated, untested). They appear in Admin > Models like image models, download to the Hugging Face cache (sub-folders only), and can be added from any LTX, Wan, MusicGen or Stable Audio repo through Admin > Add, which now recognises those pipeline classes and the transformers layout.
- media.py: MediaManager with loaders that stream every part straight onto the GPU in bf16 and cache a bf16 copy of the 19 to 23 GB fp32 text encoders; generate_video() and generate_music(); prepare_for() decides which of the image, video or music models must leave the card.
- Video tab (media_ui.py): six sizes, 2 to 10 s, steps, guidance, seed, start image for image-to-video, camera chips, examples; Upscale video runs Real-ESRGAN's compact video network over every frame and streams the result into a new MP4; Add soundtrack muxes a generated track under the clip.
- Music tab: MusicGen with length, guidance, temperature and seed, genre chips, examples, token-level progress; tracks saved as 16-bit WAV.
- video_io.py: ffmpeg from the imageio-ffmpeg wheel (no system install), streamed H.264 encoding, frame reading, thumbnails, audio muxing with fade-out, WAV writing and waveform pictures.
- History: kind filter and column, first-frame and waveform thumbnails, inline video and audio players, Reuse prompt returns to the tab that made the entry. Database columns kind, duration_s, fps, frames, sample_rate, soundtrack_path; counts_by_kind().
- Admin: Overview tiles for images / videos / tracks and seconds of media, Loaded now shows both managers, Load now and Unload work for video and music models, model status shows fps, limits and image-to-video support; the header pill shows both loaded models.
- Verified on the RTX 3090 through the Gradio API: see section 10.7 for the measured figures.

Increment 13: categories and batch generation (23 Oct 2026)

- A Category box in the Studio: images go to outputs/<category>/ instead of outputs/, the generations row carries the category, and the prompt is stored under the category in the new prompt_library table (created_at, category, prompt, kind, source, generation_id). History filters by category and shows it as a column; db.py categories / db.py prompts <category> print them.
- batch.py: generates a whole category through the running app's /generate endpoint (so every image has the usual telemetry), from a built-in prompt set (Nature: 50 prompts) or a text file, cycling prompts with fresh seeds; --model, --seed-start, --negative, --url, --list.
- Verified: 50 Nature images with Z-Image Turbo in one batch, saved to outputs/Nature/ and listed under the category in both tables.

Documentation 2.2 (22 Oct 2026)

- This document set: new section 10 (Video and Music), later sections renumbered, module reference for media.py, media_ui.py and video_io.py, requirements FR-26 to FR-29 and NFR-13 to NFR-14, workflows 4.14 and 4.15, database columns, change log.

14.2 Roadmap

Priority	Item	Notes
High	Image-to-image and inpainting	Same pipeline family; lets a chosen image be edited rather than regenerated; the ModelManager already shares components
High	Longer and higher-resolution video	LTX-Video 13B distilled (needs the 24 GB card alone, fp8), Wan 2.2 5B at 720p, and clip extension by feeding the last frame back as a start image
Medium	Lyrics and vocals	ACE-Step or YuE for songs with vocals; both need their own packages and 8 to 16 GB of VRAM
Medium	Video LoRAs and camera-control LoRAs for LTX	The LoRA registry already knows families; a video family entry would route them
High	Character consistency with IP-Adapter or reference images	Keeps a subject's identity across prompts; the usual next step after LoRAs
High	Test the Docker image on a host with WSL 2 and Docker Desktop	Written and reviewed but unbuilt on the development machine
Medium	FLUX.1 dev and ControlNet	dev is gated and non-commercial; ControlNet gives pose and depth control
Medium	SDXL refiner or hires-fix pass	Adds generated detail at 2x before Real-ESRGAN
Medium	Export history to CSV and chart timings per model	pandas is already present
Medium	Image-to-image from a History entry	The History tab already has the record and the reuse actions
Low	Full authentication	Gradio supports basic auth for the whole app; the admin password covers destructive actions only
Low	Cloud deployment profile	The Docker image is the starting point for RunPod or a Hugging Face Space

Appendix

A. Project file tree

```
ai-image-gen/
  app.py                  web application: Studio and History tabs, generation
  admin_ui.py             Admin panel: models, LoRAs, add, downloads, system, settings, logs
  registry.py             models/registry.json: settings, model overrides, LoRA metadata
  downloads.py            source inspection and background downloads
  ui_theme.py             colours, Gradio theme, CSS
  ui_common.py            shared UI helpers (choices, header, cards, tiles)
  applog.py               in-memory log for the Logs tab
  media.py                video and music engine: profiles, MediaManager, GPU coordination
  media_ui.py             Video and Music tabs
  video_io.py             ffmpeg / WAV helpers: MP4 encoding, thumbnails, muxing, waveforms
  models.py               model registry, ModelManager, LoRAs, compel
  prompt_parser.py        prompt -> settings
  sysinfo.py              system telemetry for the Admin panel
  upscaler.py             Real-ESRGAN tiled upscaling
  gpu_info.py             GPU facts and live telemetry
  db.py                   SQLite store and CLI viewer
  generate.py             command-line generator
  requirements.txt
  run-app.bat
  README.md
  Dockerfile, docker-compose.yml, .dockerignore
  generations.db          created on first generation
  models/
    RealESRGAN_x4plus.pth
    registry.json         Admin registry (created on first edit)
    cache/                bf16 / NF4 copies of converted models (auto-built)
    checkpoints/          SDXL .safetensors checkpoints (Admin › Add or by hand)
    loras/                LoRA files (Admin › Add or by hand)
  outputs/                generated PNG files
    .thumbs/              History thumbnails
  docs/                   this documentation set as PDFs
  docs-src/
    build_pdf.py          source of the documentation
    images/               figures
  .venv/                  virtual environment (not for backup)
```

B. requirements.txt

```
# Install PyTorch with CUDA first (RTX 3090 / any recent NVIDIA card):
#   pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
# Then:
#   pip install -r requirements.txt
diffusers>=0.31
transformers>=4.45
accelerate>=1.0
safetensors
huggingface_hub
gradio>=5.0
pandas
psutil>=5.9           # Admin panel: CPU, memory, disk and network stats
spandrel>=0.4          # Real-ESRGAN upscaler
peft>=0.13             # LoRA loading
compel>=2.0            # prompt weighting (word:1.3) and long prompts for SDXL models
sentencepiece          # T5 tokenizer for FLUX
protobuf               # T5 tokenizer for FLUX
bitsandbytes>=0.45     # 4-bit FLUX on 24 GB cards
requests               # Admin panel downloads (Civitai, direct URLs)
imageio                # video export (Video tab)
imageio-ffmpeg         # bundled ffmpeg binary: MP4 encoding, thumbnails, soundtrack muxing
soundfile              # WAV export (Music tab)
```

C. Command cheat sheet

All commands run from the project folder in PowerShell with `.\.venv\Scripts\python.exe`.

Task	Command
Start the app	app.py
Generate a category in batch through the app	batch.py --category Nature --count 50 (--prompts file.txt, --model, --seed-start, --list)
Categories and stored prompts	db.py categories · db.py prompts Nature
List models	generate.py --list-models
Generate from CLI	generate.py "prompt" --model juggernaut --seed 42 --count 2
Generate with a LoRA	generate.py "prompt" --model dreamshaper --lora name:0.8
Upscale a file	upscaler.py outputs\x.png 3840
List history / one record / statistics	db.py | db.py 4 | db.py stats
Parser self-test	prompt_parser.py
Model registry status	models.py
Print the Admin registry	registry.py
Inspect a model or LoRA source	downloads.py owner/repo
Start without loading a model	set FRAZILE_STARTUP_MODEL=none, then app.py
Fetch the starter LoRAs	models.py fetch-loras
GPU telemetry dump	gpu_info.py
Full system telemetry as JSON	sysinfo.py
Log in to Hugging Face (for FLUX)	.\.venv\Scripts\huggingface-cli login
Rebuild the PDFs	docs-src\build_pdf.py
Run in Docker	docker compose up --build
Check CUDA	-c "import torch; print(torch.cuda.is_available())"

D. Programmatic access to the running app

```
from gradio_client import Client
c = Client("http://127.0.0.1:7860")
images, report, records, sel, bar, hint, loaded, empty = c.predict(
    "a lighthouse, 16:9, seed 42", "", "juggernaut", [], api_name="/generate")
table, thumbs, stats, models = c.predict("lighthouse", 50, "", False, api_name="/refresh_history"
)
tiles, gpu, cpu, mem, net, power, soft = c.predict(False, api_name="/system_refresh")
tiles, activity, per_model = c.predict(api_name="/overview_refresh")
info, html, *form = c.predict("ostris/crayon_style_lora_sdxl", api_name="/check_source")
log = c.predict(200, "LoRA", "all", api_name="/logs_text")
```

E. Licences

Component	Licence
Stable Diffusion XL base 1.0	CreativeML Open RAIL++-M
Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo	CreativeML Open RAIL++-M
Playground v2.5	Playground Community License
Animagine XL 3.1	Fair AI Public License 1.0-SD
Z-Image Turbo, Qwen-Image, Qwen-Image-Lightning, FLUX.1 schnell	Apache 2.0
Real-ESRGAN weights and code	BSD 3-Clause

Component	Licence
diffusers, transformers, accelerate, peft, Gradio	Apache 2.0
compel, spandrel, bitsandbytes	MIT
psutil, PyTorch	BSD-style
Project source code	Owned by Frazile AI

Each model's licence is linked from its Hugging Face page. Check the use restrictions before commercial use.